

# Non-Delegatable Commitments

Georg Fuchsbauer<sup>1</sup>    Pranav Garimidi<sup>2</sup>    Guru-Vamsi Policharla<sup>3</sup>    Max Resnick<sup>4</sup>  
Ertem Nusret Tas<sup>2</sup>

<sup>1</sup>TU Wien

<sup>2</sup>a16z Crypto Research

<sup>3</sup>UC Berkeley

<sup>4</sup>Anza

## Abstract

Cryptographic commitments allow a party to commit to a value such that it is computationally infeasible to later open that commitment to a different value. Although they are ubiquitous, standard commitment schemes allow the committer to outsource both the generation of the commitment and the openings to a third party. This is benign for most use cases; however, when commitments serve as cryptographic attestations of work such as relaying blocks or storing data, participants can outsource the task and still claim credit, undermining the intended economic properties of the protocol. This work initiates the study of **non-delegatable commitments**, a new primitive where forming a commitment requires possession of a private key, and delegating the commitment process necessarily leaks that key. We formally define the primitive and provide a generic construction that is secure in the random oracle model given a polynomial commitment scheme. Additionally, we show how this primitive can be applied to solve a variety of mechanism design problems.

## 1 Introduction

Commitments are among the most fundamental primitives in cryptography: they allow a party to fix a hidden value now and reveal it later. However, standard commitment schemes are *delegatable* in that the committer can outsource the generation of the commitment and openings to a third party. This is fine for most applications of commitments but the designers of incentive schemes for decentralized protocols often wish to use commitments as a succinct attestation that some work was performed, or some messages were received. A standard commitment can be used as proof that *somebody* received messages or stored data but not that the person who was supposed to receive them actually did so. What mechanism designers really want is not a proof that somebody did the work but a proof that the person who was supposed to do the work (and be rewarded for it) did the work that they were supposed to do.

Concretely, suppose a bandwidth-limited *weak* verifier wishes to ascertain whether a *receiver* **R** received some large file from a *sender* **S** in a timely manner, where the verifier has a succinct commitment to the file. Importantly, the verifier cannot ask the receiver to relay the entire file and check it against the commitment, because it is bandwidth-limited. So, we need a scheme with lower communication overhead for the verifier.

Consider the following natural approach:

1. The sender **S** sends a subset of the file to the receiver **R**.
2. The receiver produces a succinct commitment *com* to the subset of the file they received and submits it to the verifier.
3. The verifier challenges **R** to open a few randomly chosen positions of the file with respect to its commitment.
4. The verifier rewards both the sender and receiver for each successful opening the receiver produces.

Unfortunately this approach has a fatal flaw. If the sender and receiver collude, they can avoid sending the file altogether, and instead **S** can simply provide **R** with the commitment and the openings corresponding to the verifier’s queries.

To address these issues, we formally define and construct *non-delegatable* (vector) commitments, which are cryptographic vector commitments with the additional guarantee that a party cannot delegate the task of creating commitments (and openings) to another party without also leaking its secret key. Given such a primitive, we can attach some value to the secret key of the receiver (e.g. the control of some funds locked in a smart contract), thereby providing an economic guarantee for data dissemination, as delegating the commitment to a third party would give that party control over the funds attached to the secret key.

More formally, let the data  $\mathbf{m}$  be a finite ordered collection of blocks  $\mathbf{m} = (m_1, \dots, m_n)$ , where each block  $m_i \in \mathcal{M}$ . **R** sends a non-delegatable commitment  $\text{com}_{\mathbf{R}}$  of the data it has received to a verifier that holds a succinct commitment  $\text{com}$  to  $\mathbf{m}$  and wishes to check that **S** indeed disseminated  $\mathbf{m}$  to **R** in a *timely* manner. Of course, there may be network failures, and **R** may not have received all of  $\mathbf{m}$ , but it should be incentivized to truthfully report what it has received. The verifier then challenges **R** to open  $k$  randomly chosen blocks of the data. Let **S** incur a cost  $c$  per message  $m_i$  sent to **R**. We now outline a reward scheme based on **R**’s attestation such that **S** sending all of  $\mathbf{m}$  and **R** attesting to  $\mathbf{m}$  is profit-maximizing for them.

Intuitively, the non-delegatable commitment scheme ensures that **R** can only attest to the blocks it has *actually* received from **S**. If **R** successfully provides all  $k$  valid openings to the verifier’s queries, then both **S** and **R** are paid  $\alpha r$ , where  $r$  is a fixed reward, and  $\alpha$  is the ratio between how many messages **R** claimed to have received, and  $n$  (the total amount of messages that were meant to be sent). A receiver never has incentive to under-report what it received since this only decreases its reward, and if a receiver claims to have heard more messages than it actually did, it risks being challenged on a missing opening and losing the entire reward for only a marginal gain otherwise. Thus, under a suitably sized reward, it is optimal for receivers to attest to exactly the messages they have. Then, given that **R** is incentivized to report exactly what it received, **S** is in turn incentivized to send all  $n$  messages to maximize its reward as long as  $r > cn$  (recall that  $c$  is **S**’s cost per message disseminated).

Two closely related notions have been studied in prior work – proofs of “individual knowledge” [DFL23] and “complete knowledge” [KBD<sup>+</sup>24], which aim to guarantee that there is a single party that knows a secret key, even in the presence of a Trusted Execution Environment or Secure Multi-Party Computation committees, which allow controlled delegation of the secret key. The key insight in these works is that cryptographic hash functions are expensive to compute in MPC (in comparison to local evaluation) or in a TEE (in comparison to an ASIC). By appropriately designing the proof of knowledge of the secret key, they are able to make the cost of delegating the proof of knowledge to an MPC/TEE infeasible if there is an upper-bound on the number of rounds of communication allowed or the time in which the proof must be generated. We view these works as complementary to ours – while they focus on preventing delegation of the secret key, we design commitments that cannot be delegated without leaking the secret key. While one could treat the message/data as the secret key in [DFL23, KBD<sup>+</sup>24], the communication and computation costs would be linear in the size of the message/data.

## 1.1 Applications

Here we sketch two examples where non-delegatable commitments can provide economic security guarantees for tasks in distributed protocols that previously only had indirect economic incentives. We emphasize that these applications are not meant to be exhaustive, but rather illustrative of the types of problems that non-delegatable commitments can help solve.

**Block Propagation in Blockchains.** In high-performance blockchains, one of the most expensive parts of participating as a validator are the network costs. For example, the current Solana validator requirement is at least 1 Gb/s Bandwidth with 10 Gb/s preferred [Sol24]. A large part of this network cost comes from block propagation, as the nodes must forward the blocks they hear to the rest of the network. Due to the large block size and number of nodes, the creator of the block may not always have the bandwidth itself

to directly broadcast the blocks it produced to the *entire* network. Instead, the creator erasure-codes its block and sends shares to a randomly chosen set of nodes designated as relays, who are in turn responsible for broadcasting their shares (much smaller than a full block) to the entire network [Anz25]. This allows amortizing bandwidth resources across the network, so that every node receives enough shares to reconstruct the block as quickly as possible.

For this scheme to work efficiently, it is crucial both that the block creator broadcasts all its shares to all the relays *and* that all the relays broadcast the shares they receive to the rest of the network in a timely manner. However, there are currently no incentives to do so. While block creators are incentivized to broadcast their shares so that they collect the transaction fees from the confirmed block, relays have no such incentive. Thus, the relays may wish to save on bandwidth costs by not dispersing the shares they receive. While this may not affect liveness in good network conditions as long as enough shares are still being broadcast, overall latency increases and the network would be impacted if a large fraction of the nodes crash due to a systemic failure.

This is the exact setting our scheme seeks to address. We can use our scheme to setup a reward scheme where relays are rewarded based on how many shares they broadcast via having nodes post commitments to whom they have heard from. In this case both the relay and receiver are rewarded proportional to how many messages are sent; so that the relay is incentivized to forward all their shares, and the receiver is incentivized to truthfully report everything they heard. Here the trusted verifier (in charge of dispensing rewards) can be a smart contract on the blockchain itself, and can verify nodes' commitments with random challenges for them to provide valid openings.

**Non-Delegatable Computation** Our non-delegatable commitment scheme can be extended to achieve the functionality of "non-delegatable computation" which provides proof that a node carried out a computational task itself rather than delegating that task to another node. At a high level, a node runs a computational task and then makes a non-delegatable commitment to the trace of the computation, and engages with a verifier to prove its possession of the intermediate values of the computation. The verifier also checks if the revealed values are correct (e.g. if the computation involves a circuit, it challenges  $k$  circuit gates and asks for the inputs and outputs to the  $k$  challenged gates, running each gate to see if the revealed values are correct). This solution does assume it is more expensive for nodes to download all (or most) of the intermediate values from a central node than to compute all the values themselves. This may be a reasonable assumption in e.g. LLM inference where communicating the entire trace may take terabytes for frontier open source models

This functionality is especially useful for protocols involving *optimistic* verification. In optimistic verification, a set of nodes check that another node correctly executed a computational task by rerunning the computation themselves and checking whether the output matches. In the case that one of the nodes disagrees about the validity of the computation, a verifier resolves the dispute via a bisection game between the two parties' committed traces. However, often these schemes only reward nodes upon detecting a fault, as without a fault, there is no evidence that any of the verifying nodes did any work.

This leads to a challenge, as if the computation is always performed correctly, there is never any fault for nodes to detect, and hence nodes never get rewarded for the work they do. This causes such schemes to rely either on incentives external to the protocol for nodes to perform verification or on "attention challenges" where faulty work is intentionally done. Instead, we would like to reward nodes directly for the work they do in rerunning computations, even when the outcome is correct. Non-delegatable computation provides exactly this functionality. Optimistic verification schemes can require nodes to post non-delegatable commitments to the work they do in re-executing the computation. Then a verifier can be a smart contract issuing challenges to nodes to provide openings to their computational traces and reward them accordingly. This strengthens the economic security of optimistic verification protocols; so that all the verifying nodes can be expected to faithfully work without external incentives.

## 2 Technical Overview

In this section, we provide an overview of how we define non-delegatable commitments (Section 5) and our constructions (Section 6) in the random oracle model.

A non-delegatable commitment scheme is a cryptographic primitive, where creating a commitment requires knowledge of the message being committed and a *secret key*. Verification of openings is done using the corresponding public key. Non-delegatable commitments satisfy standard notions of binding and hiding, but in addition they are *non-delegatable* in that it is computationally infeasible to *delegate* the creation of the commitment without also leaking the secret key. In practice, by appropriately attaching a *monetary value* to the secret key (say, by staking funds in a smart contract), we can provide an *economic* guarantee that any party creating a commitment was indeed in possession of the underlying message.

**Impossibility(?)** Clearly, non-delegatable commitments are impossible in the standard model as any functionality of the secret key can be delegated (even to an untrusted party), with full privacy, using fully homomorphic encryption [Gen09]. But no such lower bound applies to building *non-delegatable* primitives in idealized models such as the Random Oracle Model or the Generic/Algebraic Group Model [Sho97, Mau05, FKL18]. Of course, to actually deploy such a protocol, the idealized primitives would need to be *heuristically* instantiated, in which delegation is possible again, seemingly making the whole exercise pointless!

However, there is an interesting statement to be made here about the *efficiency* of such a delegation. Assuming that we prove an appropriate notion of “non-delegatability” in an idealized model (say the random oracle model), any successful delegation must inherently make *non-blackbox* use of the cryptographic hash function that is used to heuristically instantiate the random oracle. In the case of non-delegatable commitments, the delegator and delegates must engage in a secure two-party computation protocol to evaluate a function involving (potentially many) cryptographic hash function calls. For a non-delegatable commitment to be meaningful in applications such as those outlined in Section 1.1, it is sufficient to make the economic cost of delegation higher than the cost of actually carrying out the work of data dissemination. Moreover, one can even design hash functions that are inherently MPC-, FHE- and TEE-*unfriendly* and thus hard to delegate, making it substantially more expensive than to “just do the work yourself”.

Thus, a non-delegatable commitment scheme in an idealized model with *minimal overhead* for honest parties is valuable, so long as the best possible delegation protocol (for the heuristic instantiation) is more expensive than carrying out the commitment honestly.

In this work, we focus on vector commitment schemes as they are most relevant for the applications outlined in Section 1.1, but other variants such as polynomial commitment schemes are an interesting direction for future work.

**Definition.** A non-delegatable commitment scheme is a commitment scheme, satisfying the standard notions of binding (and optionally hiding).<sup>1</sup> We demand two additional properties:

- **Non-Delegatability:** it must be computationally infeasible for an adversary to “delegate” creation of a commitment without leaking its secret key.
- **Key-Hiding:** an honestly generated commitment and opening should not leak any information about the secret key. Note that this does not imply any privacy guarantees for the message.

The key-hiding property is modeled via a simulator that can produce a computationally indistinguishable transcript of commitment and openings at all points in the vector without knowledge of the secret key.

Non-delegatability is modeled via a  $k$ -soundness game between a challenger and an adversary  $\mathcal{A} = (\mathbf{S}_1, \dots, \mathbf{S}_\ell, \mathbf{R}_1, \mathbf{R}_2)$ , consisting of a set of senders and a (two-stage) receiver. To start,  $\mathbf{R}_1$  outputs a public key  $pk$ . The adversary’s goal is to have  $\mathbf{R}_1$  produce a valid commitment  $com$ , but by delegating it to  $\mathbf{S}_1, \dots, \mathbf{S}_\ell$ .  $\mathbf{R}_1$  is allowed to interact with  $\mathbf{S}$ ’s, but it must do so without leaking its secret key  $sk$ . At the end of this interaction,  $\mathbf{S}$ ’s output the message  $m \in \mathcal{M}^n$  that they claim was committed, and  $\mathbf{R}_1$  outputs  $com$  and a state  $st$ . After the commitment is created,  $\mathbf{R}_2$  is tasked with producing valid opening proofs on  $k$  randomly chosen points.  $\mathbf{R}_2$  can interact with  $\mathbf{S}$ ’s and receives  $st$  as input, and could thus know the entire message  $m$ .

<sup>1</sup>Commitments schemes with succinct commitments and openings are interesting even without the hiding property.

We say that a commitment scheme is non-delegatable if for every  $\mathcal{A}$ , whenever it succeeds in the above game, one of the following extractors must succeed (except with negligible probability):

- $\text{Ext}^{\mathbf{R}_1}$ : given access to  $\mathbf{R}_1$ 's random oracle queries, the transcript of the communication between  $\mathbf{S}_1, \dots, \mathbf{S}_\ell$  and  $\mathbf{R}_1$ , as well as the commitment com, it outputs a “large fraction” of the message  $m$ .
- $\text{Ext}^{\mathbf{S}_j}$ : given access to  $\mathbf{S}_j$ 's random oracle queries, its communication with  $\mathbf{R}_1$  and  $\mathbf{R}_2$ , the commitment com, and the final opening proofs, it outputs  $\mathbf{R}$ 's secret key sk.

This thus captures the fact that either  $\mathbf{R}_1$  knew (most of) the message, or one of the  $\mathbf{S}_j$  learned the secret key. We formally define non-delegatable (vector) commitments in Section 5.

**Construction ideas.** We build a non-delegatable commitment in the random oracle model, given a polynomial commitment scheme PC, and a vector commitment scheme VC.<sup>2</sup>

Our construction will crucially leverage the fact that queries to the random oracle must be made explicitly by a *single* party. Consider the following (obviously insecure) scheme. Given a message  $m \in \mathcal{M}^n$ , commit to it using a *hiding* Merkle tree, where the leaf nodes are  $\{(m_i, \text{sk})\}_{i \in [n]}$  and the hash function is modeled as a random oracle. To open it at a position  $i \in [n]$ , provide the leaf node and the root to the leaf path.

In the  $k$ -soundness game above, for any leaf node that can be successfully opened, the query  $(m_i, \text{sk})$  must have been made by either some  $\mathbf{S}_j$ , or  $\mathbf{R}_1$ . If they were not made by either, then that position cannot be opened, and if there are too many of these,  $\mathbf{R}_2$  will not be able to pass the challenge phase. Thus, a large fraction must be queried. Clearly, if some  $\mathbf{S}_j$  makes even one query, then the secret key is leaked. Therefore,  $\mathbf{R}_1$  must be the one to make all of them and hence “knows” a large fraction of the message. While this seems to achieve non-delegatability, it is obviously not key-hiding.

So instead of placing the secret key sk directly in all of the leaf nodes, we secret-share it. Concretely, let  $f(X)$  be a random degree- $k$  polynomial conditioned on  $f(0) = \text{sk}$  and set the leaf nodes to  $\{(m_i, f(i))\}_{i \in [n]}$ . Then, we can reveal  $k$  points in an honestly generated commitment (as done in the challenge phase) without leaking any information about the secret key. However, an adversary is not forced to use a valid secret sharing, and can insert arbitrary values. We therefore additionally commit to  $f(X)$  using a hiding polynomial commitment scheme and add a zero-knowledge proof that  $f(0) = \text{sk}$ . The verifier then demands openings of the Merkle commitment to  $(m_i, y_i)$ , as well as an opening of the polynomial commitment to  $y_i$  at position  $i$ .

**Extraction Issues.** It seems like we are essentially done since, if any  $\mathbf{S}_j$  makes any leaf node query different from the  $k$  openings of the Merkle tree leaves generated in response to a challenge, it learns the secret key by simply interpolating  $f(X)$ . Hence,  $\mathbf{R}_1$  must still make a large fraction of the queries, and we should be able to recover the message from them. While this is technically true, it's not clear that there's an *efficient* way to recover the message.

In fact, here is an explicit strategy by  $\mathbf{S}$  and  $\mathbf{R}_1$  that makes extraction difficult:

- $\mathbf{R}_1$  knows the message  $m$ , it secret-shares sk as described above and makes random oracle queries of the form  $\{(m_i, f(i))\}_{i \in [n]}$  and  $\{(\hat{m}_i, f(i))\}_{i \in [n]}$  in randomized order, for some arbitrary  $\hat{m} \in \mathcal{M}^n$ .
- $\mathbf{R}_1$  sends the answers to these queries to  $\mathbf{S}$ , which also knows the randomized order.
- $\mathbf{S}$  then completes the commitment to  $m$  by computing all of the internal nodes of the Merkle tree and sends over the root to  $\mathbf{R}_1$ , who outputs it.
- $\mathbf{S}$  outputs  $m$ .
- $\mathbf{R}_2$  can interact with  $\mathbf{S}$  to produce opening proofs and always pass the game.

Clearly  $\text{Ext}^{\mathbf{S}}$  cannot succeed. Even though  $\text{Ext}^{\mathbf{R}_1}$  can output a set of  $2n$  tuples  $\{(m_i, f(i))\}_{i \in [n]}$  and  $\{(\hat{m}_i, f(i))\}_{i \in [n]}$  within which lies the message  $m$ , it's not clear how  $\text{Ext}^{\mathbf{R}_1}$  can successfully *identify* it. A brute-force search would require checking  $2^n$  possible combinations of leaf nodes.

<sup>2</sup>Although a polynomial commitment scheme implies a vector commitment scheme, we separate the two as vector commitments can be instantiated very efficiently using Merkle trees, and it will be committing to a substantially larger input than the polynomial commitment scheme.

**Disambiguation.** We are almost at the finish line: we (extractor) have the message in our hands (among other things), and we just need a way to disambiguate between the real and dummy leaf nodes. We do so by creating a separate vector commitment  $\text{com}_{\text{VC}}$  to the leaf nodes  $\{(i, m_i)\}_{i \in [n]}$  (no secret key used) and embedding the opening proofs inside the Merkle tree such that the  $i$ -th leaf node is  $(i, m_i, f(i), \pi_{\text{VC}, i})$ . By checking the opening proof  $\pi_{\text{VC}, i}$  against the vector commitment, we are able to identify which nodes belong to  $m$ . Including  $\text{com}_{\text{VC}}$  in the non-delegatable commitment also lets one use it as a standard commitment: defining an alternative opening algorithm that only reveals  $\pi_{\text{VC}, i}$  allows for an unbounded number of openings without revealing the secret key.

**Our constructions.** In Section 6, we present a generic construction that can be instantiated with any vector commitment scheme and hiding polynomial commitment scheme. We then instantiate it using a hiding variant of KZG commitments [KZG10, KT24], which also lets us prove  $f(0) = \text{sk}$  very efficiently when key pairs are of the form  $\text{pk} = g^{\text{sk}}$ . Our instantiation is highly efficient for both the receiver (prover) and the verifier: an honest receiver only performs  $O(k) \ll n$  elliptic-curve operations and commits to the message using a Merkle commitment, where  $k$  is the number of opened leaves and  $n$  is the size of the data. The verifier can check the proof using  $O(k)$  elliptic-curve operations and  $O(1)$  pairings.

### 3 Related Work

There is a rich literature on *rational cryptography* [DHR00, HT04, ADGH06, KN08, FKN10, AL10, PH10, ACH11, GK12, GKM<sup>+</sup>13, BYK21, GKS22] that analyzes security in the presence of profit-maximizing (“rational”) participants. A major challenge for such protocols is extending the notions of game-theoretic equilibria (such as subgame-perfect equilibrium used for repeated games) to the computational setting [GK06, Hal11, GLR13]. In this context, rather than defining security as a property of the game among rational parties, the *rational protocol design* framework [GKM<sup>+</sup>13, GKTZ15, BGM<sup>+</sup>18] focuses on a “meta-game” between a protocol designer, who specifies the protocol rules for the honest parties, and an attacker.

An intermediate step towards security with rational protocol participants is to ensure that if security is ever violated, then the responsible parties can be financially penalized. This notion enables decoupling the challenge of designing cryptographic protocols for strengthening the guarantees provided by mechanism design, and that of defining game-theoretic equilibria and achieving it in a computational setting (as in [DHR00, SCW23]). Some examples of this intermediate notion include accountable threshold signatures (e.g., [BK22, BPR24b]), double-authentication-preventing signatures, and accountable assertions [PS14, BKN17, BPS17, CFS22, RKS15], which enable identifying a signer, or extracting its secret key if it creates two different signatures in the same *context*. Similarly, in the context of distributed computing protocols, accountability [BG17, SWN<sup>+</sup>21, BLR24] guarantees that if security is ever violated, then the responsible adversarial parties can be provably identified, and their escrowed stake can be burned. Although accountability has been extended to contexts beyond signatures and consensus (cf. [CFN94, GSS21, Zha25, BPR24a, BPR24c, AADG25, BPR25]), these protocols often do not support public verification of the proof identifying the malicious parties, instead relying on a trusted entity with private state for this purpose.

Our protocol fits in the same “staked” adversary paradigm, with a notable difference: in the works cited above, even if multiple parties collude to violate security, once the violation happens, a large number of them can be identified and punished up to large coalition sizes. In contrast, we restrict our attention to a setting with two classes of parties and a verifier for detecting security violations, where if both types of parties fully collude, then providing security becomes *provably* impossible. Therefore, we model our adversary as two distrustful, computationally bounded entities, which can collude to breach security, yet, would like to hide their secret keys from each other. This definition and the achieving protocol comes with its own challenges that are summarized in Section 2.

Another technique to prevent collusions among “staked” parties is whistleblowing [KGP<sup>+</sup>25], where a subset of colluders create a cryptographic proof that reveals the adversarial behavior of their counterparts. However, these techniques come with limitations, as blockchain smart contracts can help colluders retaliate against whistleblowers, requiring further solutions that require staking of protocol participants.



**Concurrent Work.** In concurrent work [DFK<sup>+</sup>25], the authors independently introduce and construct *beholder signatures*, where parties sign commitments to long messages, with the guarantee that signatures can be produced only if the signer knows the entire committed message. More formally, they show that if a “distributed” adversary [DFL23] produces a valid signature on a commitment, then there exists some sub-adversary from which one can extract the secret key, and some fraction of the committed message. Non-delegatable commitments demand a slightly weaker property (which is nevertheless sufficient for applications), where the non-delegatability game is played by  $(S_1, \dots, S_\ell)$  and  $R$ , where  $R$  has a public key  $pk$  and is instructed to create a commitment to some data  $m$ . If a commitment is produced and opened successfully, then one can extract a large fraction of the data from  $R$ , or the secret key  $sk$  from one of  $(S_1, \dots, S_\ell)$ . In particular, we do not demand that the secret key can be extracted from  $R$  (although our construction does satisfy this property).

In terms of concrete constructions, the two approaches are quite different and interesting in their own right. Fundamentally, they both use the fact that evaluating random oracles in a *distributed* setting is too slow. However, [DFK<sup>+</sup>25] uses a variant of the Fischlin transform in order to extract the message and secret key, whereas we use a Merkle tree commitment combined with verifiable secret sharing in order to prevent delegation. In terms of asymptotic efficiency when committing to data of size  $n$ , [DFK<sup>+</sup>25] has total communication  $O(\text{poly}(\lambda))$  and uses  $\tilde{O}(n)$  public key operations, whereas our work uses  $O(\text{poly}(\lambda, \log n))$  communication but only requires  $\tilde{O}(\lambda)$  public key operations

## 4 Preliminaries

Let  $e: \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_t$  denote a type-III bilinear map, where  $\mathbb{G}_1$ ,  $\mathbb{G}_2$  and  $\mathbb{G}_t$  are multiplicative groups of prime order  $p$  with generators  $g$  and  $h$  for  $\mathbb{G}_1$  and  $\mathbb{G}_2$ , respectively. We let  $[n]$  denote the set  $\{1, 2, \dots, n\}$  for positive integers  $n$ . We denote a sequence  $(x_{a_1}, \dots, x_{a_\ell})$ , where  $a_1 < \dots < a_\ell$ , by  $(x_{a_i})_{i \in [\ell]}$ , and assume that knowledge of the indices is transferred along with the tuple (this matters when  $(x_{a_1}, \dots, x_{a_\ell}) \subset [n]$  for some  $n > \ell$ ). Given a security parameter  $\lambda \in \mathbb{Z}^+$ , an event is said to happen with probability negligible in  $\lambda$ , denoted by  $\text{negl}(\lambda)$ , if its probability, as a function of  $\lambda$ , is  $\lambda^{\omega(1)}$ . A value is said to be succinct in  $n$  if it scales as  $O(\text{polylog}(n))$  as a function of  $n$ . By  $y \leftarrow M(x_1, \dots; r)$  we denote the operation of running algorithm  $M$  on inputs  $x_1, \dots$  and coins  $r$  and letting  $y$  denote the output. By  $y \leftarrow M(x_1, \dots)$ , we denote letting  $y \leftarrow M(x_1, \dots; r)$  for random  $r$ . We use  $\langle M, (M_1, \dots, M_\ell) \rangle$  to denote the transcripts of communication between the Turing machine  $M$  and  $M_1, \dots, M_\ell$  respectively. In the random oracle model (ROM),  $\text{Ext}^M$  implies that an algorithm  $\text{Ext}$  has access to the random oracle queries of a Turing machine  $M$ .

### 4.1 Non-interactive Zero-knowledge Argument of Knowledge

**Definition 1** (NIZK-AoK). A non-interactive zero-knowledge argument of knowledge, denoted by NIZK, for a relation  $\mathcal{R}$  consists of four algorithms:

- $\text{NIZK.Setup}(1^\lambda, \mathcal{R}) \rightarrow \text{crs}$ : The (randomized) setup algorithm takes a security parameter and a relation  $\mathcal{R}$  as input, and outputs a common reference string  $\text{crs}$ .
- $\text{NIZK.Prove}(\text{crs}, x, w) \rightarrow \pi$ : The (randomized) proving algorithm takes a  $\text{crs}$ , a problem instance  $x$  and a witness  $w$  as input, and outputs a zero-knowledge (ZK) proof  $\pi$ .
- $\text{NIZK.Verify}(\text{crs}, x, \pi) \rightarrow 0/1$ : The (deterministic) verification algorithm takes a  $\text{crs}$ , a problem instance  $x$  and a proof  $\pi$  as input, and outputs 1 if the proof is accepted.

A secure NIZK satisfies the following properties:

**Completeness:** For all  $\lambda \in \mathbb{N}$ , all instances/witness pairs  $(x, w) \in \mathcal{R}$ ,

$$\Pr \left[ \text{Verify}(\text{crs}, x, \pi) = 1 \mid \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda, \mathcal{R}) \\ \pi \leftarrow \text{Prove}(\text{crs}, x, w) \end{array} \right] = 1.$$

**Knowledge Soundness:** For all PPT adversaries  $\mathcal{A}$ , there exists an expected-polynomial-time extractor  $\text{Ext}$  such that

$$\Pr \left[ \begin{array}{c} \text{Verify}(\text{crs}, x, \pi) = 1 \\ \wedge (x, w) \notin \mathcal{R} \end{array} \middle| \begin{array}{c} \text{crs} \leftarrow \text{Setup}(1^\lambda) \\ (x, \pi) \leftarrow \mathcal{A}(1^\lambda, \text{crs}) \\ w \leftarrow \text{Ext}^\mathcal{A}(\text{crs}, x, \pi) \end{array} \right] \leq \text{negl}(\lambda)$$

**Statistical (Computational) Zero-knowledge (non-adaptive):** There exists a PPT simulator  $\mathcal{S}$  such that for all  $\lambda \in \mathbb{N}$ , for all  $(x, w) \in \mathcal{R}$  and all unbounded (PPT) adversaries  $\mathcal{A}$ ,

$$\Pr \left[ \mathcal{A}(\text{crs}_b, \pi_b) = b \middle| \begin{array}{c} b \leftarrow \{0, 1\} \\ \text{crs}_0 \leftarrow \text{Setup}(1^\lambda) \\ \pi_0 \leftarrow \text{Prove}(\text{crs}_0, x, w), \\ (\text{crs}_1, \pi_1) \leftarrow \mathcal{S}(1^\lambda, x) \end{array} \right] \leq \frac{1}{2} + \text{negl}(\lambda).$$

## 4.2 Vector Commitments

**Definition 2** (Vector Commitments). A vector commitment (VC) scheme for message space  $\mathcal{M}$  is defined by the following algorithms:

- $\text{VC.Setup}(1^\lambda, 1^n) \rightarrow \text{crs}$ : The setup algorithm takes a security parameter  $\lambda$  and a vector length  $n$  as input and outputs a CRS, which is an implicit input to all subsequent algorithms.
- $\text{VC.Commit}((m_i)_{i \in [n]} \in \mathcal{M}^n) \rightarrow (\text{com}_{\text{VC}}, \text{aux}_{\text{VC}})$ : The (randomized) commitment algorithm takes a sequence of  $n$  messages from some set  $\mathcal{M}$  as input and outputs a commitment  $\text{com}_{\text{VC}}$  and auxiliary information  $\text{aux}_{\text{VC}}$ .
- $\text{VC.Open}(\text{com}_{\text{VC}}, \text{aux}_{\text{VC}}, i \in [n]) \rightarrow (m_i, \pi_{\text{VC}})$ : The (randomized) opening algorithm takes  $\text{com}_{\text{VC}}$ ,  $\text{aux}_{\text{VC}}$  and an index  $i$  as input and outputs the message at index  $i$  along with a corresponding opening proof  $\pi_{\text{VC}}$ .
- $\text{VC.Verify}(\text{com}_{\text{VC}}, i, m_i, \pi_{\text{VC}}) \rightarrow 0/1$ : The (deterministic) verification algorithm takes a commitment  $\text{com}_{\text{VC}}$ , an index  $i \in [n]$ , a claimed value  $m_i$  at index  $i$ , and an opening proof  $\pi_{\text{VC}}$  as input and outputs 1 if the proof verifies.

A secure VC satisfies correctness, binding, position-binding and hiding.

**VC Correctness:** For all  $\lambda \in \mathbb{N}$ ,  $n = \text{poly}(\lambda)$ , messages  $(m_i)_{i \in [n]} \in \mathcal{M}^n$ , and indices  $i \in [n]$ , it holds that

$$\Pr \left[ \text{VC.Verify}(\text{com}_{\text{VC}}, i, m_i, \pi_{\text{VC}}) = 1 \middle| \begin{array}{c} \text{crs} \leftarrow \text{VC.Setup}(1^\lambda, 1^n) \\ (\text{com}_{\text{VC}}, \text{aux}_{\text{VC}}) \leftarrow \text{VC.Commit}((m_i)_{i \in [n]}) \\ (m_i, \pi_{\text{VC}}) \leftarrow \text{VC.Open}(\text{com}_{\text{VC}}, \text{aux}_{\text{VC}}, i) \end{array} \right] > 1 - \text{negl}(\lambda)$$

**VC Binding:** For all  $\lambda \in \mathbb{N}$ ,  $n = \text{poly}(\lambda)$ , and PPT adversaries  $\mathcal{A}$ ,

$$\Pr \left[ \begin{array}{c} \text{VC.Commit}((m_i)_{i \in [n]}; r) \\ = \text{VC.Commit}((m'_i)_{i \in [n]}; r') \\ \wedge (m_i)_{i \in [n]} \neq (m'_i)_{i \in [n]} \end{array} \middle| \begin{array}{c} \text{crs} \leftarrow \text{VC.Setup}(1^\lambda, 1^n) \\ ((m_i)_{i \in [n]}, (m'_i)_{i \in [n]}, r, r') \leftarrow \mathcal{A}(\text{crs}) \end{array} \right] \leq \text{negl}(\lambda)$$

**VC Position-Binding:** For all  $\lambda \in \mathbb{N}$ ,  $n = \text{poly}(\lambda)$ , and PPT adversaries  $\mathcal{A}$ ,

$$\Pr \left[ \begin{array}{c} \text{VC.Verify}(\text{com}_{\text{VC}}, i, m', \pi'_{\text{VC}}) = 1 \\ \wedge \text{VC.Verify}(\text{com}_{\text{VC}}, i, m, \pi_{\text{VC}}) = 1 \\ \wedge m \neq m' \end{array} \middle| \begin{array}{c} \text{crs} \leftarrow \text{VC.Setup}(1^\lambda, 1^n) \\ (\text{com}_{\text{VC}}, i, m, m', \pi_{\text{VC}}, \pi'_{\text{VC}}) \leftarrow \mathcal{A}(\text{crs}) \end{array} \right] \leq \text{negl}(\lambda)$$



**VC Statistical (Computational) Hiding:** For all  $\lambda \in \mathbb{N}$ ,  $n = \text{poly}(\lambda)$ , proper subsets  $S \subset [n]$  (of size  $< n$ ), messages  $(m_i^0)_{i \in [n]}, (m_i^1)_{i \in [n]} \in (\mathcal{M})^n$  such that  $m_i^0 = m_i^1$  for all  $i \in S$ , and unbounded (PPT) adversaries  $\mathcal{A}$ ,

$$\Pr \left[ \mathcal{A}(\text{crs}, \text{com}_b, (m_x^b, \pi_x^b)_{x \in S}) = b \mid \begin{array}{l} b \leftarrow \{0, 1\} \\ \text{crs} \leftarrow \text{VC.Setup}(1^\lambda, 1^n) \\ (\text{com}_0, \text{aux}_0) \leftarrow \text{VC.Commit}(\text{crs}, (m_i^0)_{i \in [n]}) \\ \{(m_x^0, \pi_x^0) \leftarrow \text{VC.Open}(\text{com}_0, \text{aux}_0, x)\}_{x \in S} \\ (\text{com}_1, \text{aux}_1) \leftarrow \text{VC.Commit}(\text{crs}, (m_i^1)_{i \in [n]}) \\ \{(m_x^1, \pi_x^1) \leftarrow \text{VC.Open}(\text{com}_1, \text{aux}_1, x)\}_{x \in S} \end{array} \right] \leq \frac{1}{2} + \text{negl}(\lambda)$$

**Remark 1.** In our construction (Section 6), we use a Merkle tree, where each input message  $m_i$  is concatenated with a fresh, uniformly random value  $r_i \leftarrow \{0, 1\}^\lambda$ . This construction satisfies binding, position-binding, and computational hiding, when the hash function is modeled as a random oracle. For this VC, we use the notation ‘MT’ in place of VC in the algorithms. In this case, there is no CRS,  $\text{aux}_{\text{MT}}$  corresponds to the Merkle tree, the commitment is the Merkle root, and the opening proofs consist of the vector length  $n$  along with a subset of Merkle tree’s inner nodes. The randomly sampled values  $r_i$  attached to the messages  $m_i$  are provided as part of the opening proofs  $\pi_i$  to enable verification.

### 4.3 Polynomial Commitments

**Definition 3** (Polynomial Commitments). A univariate polynomial commitment scheme PC is characterized by the following algorithms:

- $\text{PC.Setup}(1^\lambda, 1^d) \rightarrow \text{crs}$ : The setup algorithm takes a security parameter and a degree bound  $d$  as input and outputs a CRS, which is an implicit input to all subsequent algorithms.  $\text{crs}$  specifies a finite field  $\mathbb{Z}_p$  of prime order  $p$ .
- $\text{PC.Commit}(f(X) \in \mathbb{Z}_p^{\leq d}[X]) \rightarrow (\text{com}_{\text{PC}}, \text{aux}_{\text{PC}})$ : The (randomized) commitment algorithm takes a degree  $\leq d$  polynomial  $f(X)$  as input and outputs a commitment  $\text{com}_{\text{PC}}$  and auxiliary information  $\text{aux}_{\text{PC}}$ .
- $\text{PC.Open}(\text{com}_{\text{PC}}, \text{aux}_{\text{PC}}, x \in \mathbb{Z}_p) \rightarrow (y \in \mathbb{Z}_p, \pi_{\text{PC}})$ : The (randomized) opening algorithm takes  $\text{com}_{\text{PC}}$ ,  $\text{aux}_{\text{PC}}$  and an opening point  $x \in \mathbb{Z}_p$  as input and outputs the opening value  $y \in \mathbb{Z}_p$  and a proof  $\pi_{\text{PC}}$ .
- $\text{PC.Verify}(\text{com}_{\text{PC}}, x, y, \pi_{\text{PC}}) \rightarrow 0/1$ : The (deterministic) verification algorithm takes a commitment  $\text{com}_{\text{PC}}$ , an opening point  $x$ , a claimed value  $y$ , and an opening proof  $\pi_{\text{PC}}$ ; it outputs 1 if  $\pi_{\text{PC}}$  verifies with respect to  $\text{com}_{\text{PC}}$ ,  $x$ , and  $y$ .

A secure PC satisfies correctness, binding, evaluation-binding and hiding. We additionally require the PC to be extractable as defined below.

**PC Correctness:** For all  $\lambda \in \mathbb{N}$ ,  $d = \text{poly}(\lambda)$ , polynomials  $f(X) \in \mathbb{Z}_p^{\leq d}[X]$ , and points  $x \in \mathbb{Z}_p$ , it holds that

$$\Pr \left[ \text{PC.Verify}(\text{com}_{\text{PC}}, x, y, \pi_{\text{PC}}) = 1 \mid \begin{array}{l} \text{crs} \leftarrow \text{PC.Setup}(1^\lambda, 1^d) \\ (\text{com}_{\text{PC}}, \text{aux}_{\text{PC}}) \leftarrow \text{PC.Commit}(f(X)) \\ (y, \pi_{\text{PC}}) \leftarrow \text{PC.Open}(\text{com}_{\text{PC}}, \text{aux}_{\text{PC}}, x) \end{array} \right] > 1 - \text{negl}(\lambda)$$

**PC Binding:** For all  $\lambda \in \mathbb{N}$ ,  $d = \text{poly}(\lambda)$ , and PPT adversaries  $\mathcal{A}$ ,

$$\Pr \left[ \begin{array}{l} \text{PC.Commit}(f(X); r) = \text{PC.Commit}(f'(X); r') \\ \wedge f(X) \neq f'(X) \end{array} \mid \begin{array}{l} \text{crs} \leftarrow \text{PC.Setup}(1^\lambda, 1^d) \\ (f(X), f'(X), r, r') \leftarrow \mathcal{A}(\text{crs}) \end{array} \right] = \text{negl}(\lambda)$$

**PC Evaluation-Binding:** For all  $\lambda \in \mathbb{N}$ ,  $d = \text{poly}(\lambda)$ , and PPT adversaries  $\mathcal{A}$ ,

$$\Pr \left[ \begin{array}{l} \text{VC.Verify}(\text{com}_{\text{PC}}, x, y', \pi'_{\text{PC}}) = 1 \\ \wedge \text{VC.Verify}(\text{com}_{\text{PC}}, x, y, \pi_{\text{PC}}) = 1 \\ \wedge y \neq y' \end{array} \mid \begin{array}{l} \text{crs} \leftarrow \text{PC.Setup}(1^\lambda, 1^d) \\ (\text{com}_{\text{PC}}, x, y, y', \pi_{\text{PC}}, \pi'_{\text{PC}}) \leftarrow \mathcal{A}(\text{crs}) \end{array} \right] = \text{negl}(\lambda)$$

**PC Statistical (Computational) Hiding:** For all  $\lambda \in \mathbb{N}$ ,  $d = \text{poly}(\lambda)$ , subsets  $S \subset \mathbb{Z}_p$  of size  $\leq d$ , polynomials  $f_0(X), f_1(X) \in \mathbb{Z}_p^{\leq d}[X]$  such that  $f_0(x) = f_1(x)$  for all  $x \in S$ , and unbounded (PPT) adversaries  $\mathcal{A}$ ,

$$\Pr \left[ \mathcal{A}(\text{crs}, \text{com}_b, (f_b(x), \pi_x^b)_{x \in S}) = b \mid \begin{array}{l} b \leftarrow \{0, 1\} \\ \text{crs} \leftarrow \text{PC.Setup}(1^\lambda, 1^d) \\ (\text{com}_0, \text{aux}_0) \leftarrow \text{PC.Commit}(f_0(X)) \\ \{(f_0(x), \pi_x^0) \leftarrow \text{PC.Open}(\text{com}_0, \text{aux}_0, x)\}_{x \in S} \\ (\text{com}_1, \text{aux}_1) \leftarrow \text{PC.Commit}(f_1(X)) \\ \{(f_1(x), \pi_x^1) \leftarrow \text{PC.Open}(\text{com}_1, \text{aux}_1, x)\}_{x \in S} \end{array} \right] \leq \frac{1}{2} + \text{negl}(\lambda)$$

**PC Extractability:** The opening and verification algorithms of the PCS scheme constitute a non-interactive proof system for the language

$$\{(\text{com}_{\text{PC}}, x, y) \mid \exists (f(X), r) \in \mathbb{Z}_p^{\leq d} \times \{0, 1\}^* : \text{PC.Commit}(f(X); r) = \text{com}_{\text{PC}} \wedge f(x) = y\}$$

that satisfies knowledge soundness. Note that a PCS scheme satisfies evaluation-binding if it is binding and extractable.

#### 4.4 Hiding and Extractable KZG Polynomial Commitments

We briefly recall a hiding and extractable variant of the KZG polynomial commitment scheme [KZG10, ZGK<sup>+</sup>17, KT24]. We also state algorithms that allow for opening (and verifying) many positions simultaneously.

- $\text{KZG.Setup}(1^\lambda, 1^d) \rightarrow \text{crs}$ : The setup algorithm takes a security parameter  $\lambda$  and a degree bound  $d$  as input, generates a bilinear group  $(p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_t, e, g, h)$ , samples  $\tau, \eta \leftarrow \mathbb{Z}_p$ , and outputs the following as public parameters:

$$(g^\tau, g^{\tau^2}, \dots, g^{\tau^d}, g^\eta, h^\tau, h^\eta) \in (\mathbb{G}_1)^{d+1} \times (\mathbb{G}_2)^2$$

- $\text{KZG.Commit}(f(X)) \rightarrow (\text{com}_{\text{KZG}}, \text{aux}_{\text{KZG}})$ : The commitment algorithm takes a polynomial  $f(X) = \sum_{j=0}^{\deg(f)} f_j X^j$ , with  $\deg(f) \leq d$  as input, samples a random integer  $r \leftarrow \mathbb{Z}_p$ , and outputs  $\text{com}_{\text{KZG}} = g^{f(\tau) + \eta r}$  and  $\text{aux}_{\text{KZG}}$  computed as follows:

$$\text{com}_{\text{KZG}} = (g^\eta)^r \cdot \prod_{j=0}^{\deg(f)} (g^{\tau^j})^{f_j} \quad \text{aux}_{\text{KZG}} = (f(X), r)$$

- $\text{KZG.Open}(\text{com}_{\text{KZG}}, \text{aux}_{\text{KZG}}, x) \rightarrow (f(x), \pi_{\text{KZG}})$ : The opening algorithm takes  $\text{com}_{\text{KZG}}$ , and  $\text{aux}_{\text{KZG}} = (f(X), r)$  as well as an opening point  $x \in \mathbb{Z}_p$  as input, samples a random integer  $s \leftarrow \mathbb{Z}_p$ , and outputs the opening proof

$$\pi_{\text{KZG}} := ((g^\eta)^s \cdot g^{f_x(\tau)}, g^{(r+sx)} / (g^\tau)^s),$$

where  $f_x(X) := \frac{f(X) - f(x)}{X - x}$  is the quotient polynomial, and  $g^{f_x(\tau)}$  is computed as shown above using the polynomial  $f_x(X)$ .

- $\text{KZG.Verify}(\text{com}_{\text{KZG}}, x, y, \pi_{\text{KZG}}) \rightarrow 0/1$ : The verification algorithm takes a commitment  $\text{com}_{\text{KZG}}$ , an opening point  $x \in \mathbb{Z}_p$ , a claimed value  $y \in \mathbb{Z}_p$ , and an opening proof  $\pi_{\text{KZG}} := (\pi_{\text{KZG}}^1, \pi_{\text{KZG}}^2) \in (\mathbb{G}_1)^2$ . It outputs 1 iff

$$e(\text{com}_{\text{KZG}} / g^y, h) = e(\pi_{\text{KZG}}^1, h^\tau / h^x) \cdot e(\pi_{\text{KZG}}^2, h^\eta)$$

- $\text{KZG.BatchOpen}(\text{com}_{\text{KZG}}, \text{aux}_{\text{KZG}}, (x_i)_{i \in [k]}) \rightarrow ((f(x_i))_{i \in [k]}, \pi_{\text{KZG}})$ : The batch opening algorithm takes  $\text{com}_{\text{KZG}}$ ,  $\text{aux}_{\text{KZG}} = (f(X), r)$  and a sequence of opening points  $(x_i)_{i \in [k]} \in (\mathbb{Z}_p)^k$  as input, samples a

random integer  $s \leftarrow \mathbb{Z}_p$ , and finds the degree- $(k-1)$  polynomial  $r(X)$  such that  $r(x_i) = f(x_i)$  for  $i \in [k]$ , and the degree- $k$  polynomial  $z(x) = \prod_{i \in [k]} (X - x_i)$ . It then outputs the opening proof

$$\pi_{\text{KZG}} := ((g^n)^s \cdot g^{f_{(x_i)_{i \in [k]}}(\tau)}, g^r / (g^{z(\tau)})^s),$$

where  $f_{(x_i)_{i \in [k]}}(X) := \frac{f(X) - r(X)}{\prod_{i \in [k]} (X - x_i)}$  is the quotient polynomial.

- $\text{KZG.BatchVerify}(\text{com}_{\text{KZG}}, (x_i)_{i \in [k]}, (y_i)_{i \in [k]}, \pi_{\text{KZG}}) \rightarrow 0/1$ : The verification algorithm takes a commitment  $\text{com}_{\text{KZG}}$ , a sequence of opening points  $(x_i)_{i \in [k]} \in (\mathbb{Z}_p)^k$ , claimed values  $(y_i)_{i \in [k]} \in (\mathbb{Z}_p)^k$ , and an opening proof  $\pi_{\text{KZG}} := (\pi_{\text{KZG}}^1, \pi_{\text{KZG}}^2)$ . It outputs 1 iff

$$e(\text{com}_{\text{KZG}} / g^{r(\tau)}, h) = e(\pi_{\text{KZG}}^1, h^{z(\tau)}) \cdot e(\pi_{\text{KZG}}^2, h^n)$$

Before we state the security guarantees of the scheme, we recap the  $q$ -Discrete-Logarithm assumption.

**Definition 4.** The  $q$ -Discrete-Logarithm assumption, denoted by  $q$ -DLOG and parameterized by  $q \in \mathbb{N}$ , states that for any PPT adversary  $\mathcal{A}$ ,

$$\Pr \left[ \tau' = \tau \mid \tau \leftarrow \mathbb{Z}_p, \tau' \leftarrow \mathcal{A}(g, g^\tau, \dots, g^{\tau^q}, h, h^\tau, \dots, h^{\tau^q}) \right] = \text{negl}(\lambda)$$

Extractability of the PCS scheme is shown in the Algebraic Group Model [FKL18], which assumes that any adversary that outputs a group element also outputs its representation as a linear combination of the group elements previously received. The following was proven in [KT24, Sections 3.5.3, B.2.2].

**Theorem 1.** The PCS scheme above satisfies completeness, statistical hiding, binding under the  $d$ -DLOG assumption, and extractability under the  $(d+1)$ -DLOG assumption in the Algebraic Group Model.

## 5 Definition

In this section, we formally define Non-Delegatable Commitments.

**Definition 5** (Non-Delegatable Commitments). A non-delegatable commitment scheme is a tuple of six algorithms (KeyGen, Setup, Commit, Open, Verify), characterized by a key-pair relation  $\mathcal{R}$ , and message space  $\mathcal{M}$ :

- $\text{KeyGen}(1^\lambda) \rightarrow (\text{pk}, \text{sk})$ : The key generation algorithm takes as input a security parameter and outputs a key pair  $(\text{pk}, \text{sk}) \in \mathcal{R}$ .
- $\text{Setup}(1^\lambda, 1^n, 1^k) \rightarrow \text{crs}$ : The setup algorithm takes as input a security parameter, the message length  $n$  and a soundness parameter  $k$ , and outputs a common reference string, which is an implicit input to all remaining algorithms.
- $\text{Commit}(\text{sk}, m) \rightarrow (\text{com}, \text{aux})$ : The commitment algorithm is a randomized algorithm that takes as input the secret key and a message  $m \in \mathcal{M}^n$ , and outputs a commitment  $\text{com}$ , along with auxiliary information  $\text{aux}$  that will be used to open the commitment.
- $\text{Open}(\text{sk}, \text{com}, \text{aux}, i) \rightarrow (m_i, \pi)$ : The opening algorithm is a randomized algorithm that takes as input the secret key, a commitment, auxiliary information  $\text{aux}$  and an index  $i \in [n]$ , and outputs the  $i$ -th entry  $m_i \in \mathcal{M}$  and a proof  $\pi$ .
- $\text{Verify}(\text{pk}, \text{com}, i, m_i, \pi) \rightarrow 0/1$ : The verification algorithm is a deterministic algorithm that takes as input a public key  $\text{pk}$ , commitment  $\text{com}$ , index  $i \in [n]$ , message  $m_i$ , and proof  $\pi$ , and outputs 1 if  $\pi$  is a valid opening proof for the message at the  $i$ -th position with respect to  $\text{com}$ .

A non-delegatable commitment scheme satisfies correctness if for all  $\lambda \in \mathbb{N}$ ,  $n = \text{poly}(\lambda)$ ,  $k \leq n$  for all messages  $m \in \mathcal{M}^n$  and all indices  $i \in [n]$ , we have:

$$\Pr \left[ \text{Verify}(\text{pk}, \text{com}, i, m_i, \pi) = 1 \mid \begin{array}{l} (\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(1^\lambda) \\ \text{crs} \leftarrow \text{Setup}(1^\lambda, 1^n, 1^k) \\ (\text{com}, \text{aux}) \leftarrow \text{Commit}(\text{sk}, m) \\ (m_i, \pi) \leftarrow \text{Open}(\text{sk}, \text{com}, \text{aux}, i) \end{array} \right] = 1$$

A non-delegatable commitment scheme is said to be secure if in addition to the standard notions of Binding and Position-Binding (Definition 6), it satisfies Non-Delegatability (Definition 8) and Key-Hiding (Definition 9). One can optionally demand the standard notion of hiding in commitments but it is not required in our applications.

**Definition 6** (Binding and Position-Binding). A non-delegatable commitment scheme is said to be binding and position binding if for all PPT adversaries  $\mathcal{A}$

$$\Pr \left[ \begin{array}{l} \text{Commit}(\text{sk}, m; r) = \text{Commit}(\text{sk}', m'; r') \\ \wedge m \neq m' \end{array} \mid \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda, 1^n, 1^k) \\ (\text{sk}, \text{sk}', m, m' \in \mathcal{M}^n, r, r') \leftarrow \mathcal{A}(\text{crs}) \end{array} \right] \leq \text{negl}(\lambda),$$

$$\Pr \left[ \begin{array}{l} \text{Verify}(\text{pk}, \text{com}, i, m_i, \pi) = 1 \\ \wedge \text{Verify}(\text{pk}, \text{com}, i, m'_i, \pi') = 1 \\ \wedge m_i \neq m'_i \end{array} \mid \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda, 1^n, 1^k) \\ (\text{pk}, \text{com}, i, m_i, \pi, m'_i, \pi') \leftarrow \mathcal{A}(\text{crs}) \end{array} \right] \leq \text{negl}(\lambda)$$

**Definition 7** (Succinctness). Both  $|\text{com}|$  and  $|\pi|$  are  $O(\text{polylog}(n))$ , and the Verify algorithm runs in time  $O(\text{polylog}(n))$ .

**Non-delegatability.** This notion demands that it is computationally infeasible for an adversary to “delegate” the creation of a commitment without leaking its secret key. We formally model it as a game (Fig. 1) between a challenger and adversary  $\mathcal{A} = (\mathbf{S}_1, \dots, \mathbf{S}_\ell, \mathbf{R}_1, \mathbf{R}_2)$ . To start,  $(\mathbf{S}_1, \dots, \mathbf{S}_\ell)$  collectively output a message  $m \in \mathcal{M}^n$  and  $\mathbf{R}_1$  outputs a public key  $\text{pk}$ . The goal is to have  $\mathbf{R}_1$  produce a commitment  $\text{com}$  to  $m$ , without actually “knowing” (a large part of) the underlying message.  $\mathbf{R}_1$  is allowed to coordinate with  $(\mathbf{S}_1, \dots, \mathbf{S}_\ell)$  but it must do so without leaking its secret key  $\text{sk}$ . After the commitment is created,  $\mathbf{R}_1$  can pass arbitrary state to  $\mathbf{R}_2$ , which must successfully produce valid opening proofs for  $k$  randomly chosen points in order to pass the game.  $\mathbf{R}_2$  is allowed to interact arbitrarily with  $(\mathbf{S}_1, \dots, \mathbf{S}_\ell)$  but again, must not leak the secret key  $\text{sk}$ .

To model these knowledge requirements more formally, we demand that one of the two must be true:

- **Data is Known** (at time of committing). There exists an efficient extractor that is able to recover a sufficiently large fraction of the message, given only the transcript of the communication  $\langle \mathbf{R}_1, (\mathbf{S}_1, \dots, \mathbf{S}_\ell) \rangle$ , along with any random oracle queries made by  $\mathbf{R}_1$ . This captures the case that the data is “known” to the receiver when it produces the commitment.
- **Key is Leaked.** There exists  $j \in [\ell]$ , such that an efficient extractor can recover the secret key corresponding to  $\text{pk}$ , given only the opening proofs and the transcripts of communication  $\langle \mathbf{S}_j, \mathbf{R}_1 \rangle$  and  $\langle \mathbf{S}_j, \mathbf{R}_2 \rangle$ , along with any random oracle queries made by  $\mathbf{S}_j$ .

We remark that if the latter case is true, then even if  $(\mathbf{S}_1, \dots, \mathbf{S}_\ell)$  offered a secure MPC service to  $(\mathbf{R}_1, \mathbf{R}_2)$ , it must be that there was at least one party  $\mathbf{S}_j$  that learned the secret key. If  $(\mathbf{R}_1, \mathbf{R}_2)$  are rational and have put up some collateral against their public key, then it is not in their best interest to engage in such a protocol. Therefore, the former case must be true, and the data must have been “known” to  $\mathbf{R}_1$  at the time of committing.

**Definition 8** (Non-Delegatability). A non-delegatable commitment scheme is  $\mu(k)$ -sound, if for all sufficiently large  $n$  and all PPT adversaries  $\mathcal{A} = (\mathbf{S}_1, \dots, \mathbf{S}_\ell, \mathbf{R}_1, \mathbf{R}_2)$ , there exists a polynomial-time extractor  $\text{Ext}$  such that  $\mathcal{A}$  can win the  $\mu(k)$ -soundness game defined in Fig. 1 at most with negligible probability.

(Recall that  $\langle M, (M_1, \dots, M_\ell) \rangle$  denotes the transcript between  $M$  and  $(M_1, \dots, M_\ell)$  and that  $\text{Ext}^M$  implies that  $\text{Ext}$  has black-box access to the Turing machine  $M$  and to  $M$ ’s random oracle queries.)

$\mu(k)$ -Soundness Game (in the ROM)

1. Run  $\text{crs} \leftarrow \text{Setup}(1^\lambda, 1^n, 1^k)$  and send  $\text{crs}$  to  $\mathcal{A} = (\mathbf{S}_1, \dots, \mathbf{S}_\ell, \mathbf{R}_1, \mathbf{R}_2)$ .
2.  $(\mathbf{S}_1, \dots, \mathbf{S}_\ell)$  and  $\mathbf{R}_1$  engage in an interactive protocol (during which they can query the random oracle) at the end of which
  - $(\mathbf{S}_1, \dots, \mathbf{S}_\ell)$  output a message  $m \in (\mathcal{M})^n$
  - $\mathbf{R}_1$  outputs a public key  $\text{pk}$ , a commitment  $\text{com}$  and state  $\text{st}$
3. Sample a uniformly random  $k$ -sized subset  $\text{chal} \subseteq [n]$
4.  $\mathbf{R}_2$ , on input  $(\text{st}, \text{pk}, \text{com}, \text{chal})$ , engages in an interactive protocol with  $(\mathbf{S}_1, \dots, \mathbf{S}_\ell)$  (during which they can query the random oracle) at the end of which
  - $\mathbf{R}_2$  outputs  $\{(m'_i, \pi_i)\}_{i \in \text{chal}}$
5. Run the extractors
  - $m'' \leftarrow \text{Ext}^{\mathbf{R}_1}(\text{pk}, \text{com}, \langle \mathbf{R}_1, (\mathbf{S}_1, \dots, \mathbf{S}_\ell) \rangle)$
  - $\text{sk}_j \leftarrow \text{Ext}^{\mathbf{S}_j}(\text{pk}, \text{com}, \langle \mathbf{S}_j, \mathbf{R}_1 \rangle, \langle \mathbf{S}_j, \mathbf{R}_2 \rangle, \{(m'_i, \pi_i)\}_{i \in \text{chal}})$  for all  $j \in [\ell]$
6. Output 1 if all of the following hold:
  - $m'_i = m_i$  for all  $i \in \text{chal}$
  - $\text{Verify}(\text{pk}, \text{com}, i, m_i, \pi_i) = 1$  for all  $i \in \text{chal}$
  - $|\{i \in [n] \mid m_i = m''_i\}| \leq (1 - \mu(k))n$
  - $(\text{pk}, \text{sk}_j) \notin \mathcal{R}$  for all  $j \in [\ell]$

Figure 1: The soundness game for a non-delegatable commitment with adversary  $\mathcal{A} = (\mathbf{S}_1, \dots, \mathbf{S}_\ell, \mathbf{R}_1, \mathbf{R}_2)$ .

**Key-hiding.** This notion differs from the standard notion of *hiding* in that it only guarantees privacy for the committer's secret key but not for the committed message itself. In particular, it guarantees that,  $k$  opening proofs will (computationally) not reveal anything about the secret key.

**Definition 9** (Key-Hiding). *A non-delegatable commitment scheme is  $k$ -key-hiding if there exists a simulator  $\mathcal{S}$  such that for all messages  $m \in \mathcal{M}^n$ , and all key pairs sampled as  $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(1^\lambda)$ , the following distributions are computationally indistinguishable:*

$$\{\text{pk}, \text{crs}, \text{com}, \text{chal}, m, \{\pi_i\}_{i \in \text{chal}}\} \approx_c \mathcal{S}(\text{pk}, m)$$

where

- $\text{crs} \leftarrow \text{Setup}(1^\lambda, 1^n, 1^k)$
- $(\text{com}, \text{aux}) \leftarrow \text{Commit}(\text{sk}, m)$
- $\text{chal}$  is a  $k$ -sized subset of  $[n]$  sampled uniformly at random
- $\{(m_i, \pi_i) \leftarrow \text{Open}(\text{sk}, \text{com}, \text{aux}, i)\}_{i \in \text{chal}}$

**Remark 2.** One could define a fully non-interactive scheme where the commitment always comes with a proof consisting of openings at positions determined via the Fiat-Shamir transform. Indeed, by demanding that the challenge is uniform, we can always recover the non-interactive version. We opt for an interactive definition, as it allows mechanism designers to dynamically tune the security guarantee needed, by increasing/decreasing the

number of queries made. For instance, one may choose the number of queries a party must reveal based on the amount of collateral they have put up – a larger loss may be a large deterrent for cheating and therefore can be coupled with a lower probability of getting caught. Explicitly allowing this flexibility can result in more efficient protocols.

Our construction in Section 6 is a public-coin protocol, and can thus be made non-interactive using the Fiat-Shamir transformation.

## 6 Construction

In this section, we give two constructions of non-delegatable commitments in the random oracle model.

### 6.1 Generic Construction

Our first construction is generic over any vector commitment scheme VC and any hiding polynomial commitment scheme PC (not necessarily extractable) as defined in Section 4, and any efficiently sampleable key-pair relation  $\mathcal{R}$ . For ease of presentation, we assume the witness space of  $\mathcal{R}$  is contained in  $\mathbb{Z}_p$  (the corresponding field of PC) and define the following relation:

$$\mathcal{R}_{\text{NIZK}} = \left\{ ((pk, \text{com}_{\text{PC}}), (sk, f(X), r)) \mid \begin{array}{l} (pk, sk) \in \mathcal{R} \\ f(X) \in \mathbb{Z}_p^{\leq k}[X] \\ \text{com}_{\text{PC}} = \text{PC.Commit}(f(X); r) \\ f(0) = sk \end{array} \right\}$$

- $\text{KeyGen}(1^\lambda)$  :
  - sample and output  $(pk, sk) \leftarrow \mathcal{R}$ .
- $\text{Setup}(1^\lambda, 1^n, 1^k)$  :
  - sample  $\text{crs}_{\text{PC}} \leftarrow \text{PC.Setup}(1^\lambda, 1^k)$
  - sample  $\text{crs}_{\text{VC}} \leftarrow \text{VC.Setup}(1^\lambda, 1^n)$
  - sample  $\text{crs}_{\text{NIZK}} \leftarrow \text{NIZK.Setup}(1^\lambda, \mathcal{R}_{\text{NIZK}})$
  - output  $\text{crs} := (\text{crs}_{\text{PC}}, \text{crs}_{\text{VC}}, \text{crs}_{\text{NIZK}})$ .
- $\text{Commit}(sk, m)$  :
  - commit to  $m \in \mathcal{M}^n$  with VC and compute opening proofs for all points:

$$\begin{aligned} (\text{com}_{\text{VC}}, \text{aux}_{\text{VC}}) &\leftarrow \text{VC.Commit}(m) \\ \{(m_i, \pi_{\text{VC}, i}) &\leftarrow \text{VC.Open}(\text{com}_{\text{VC}}, \text{aux}_{\text{VC}}, i)\}_{i \in [n]} \end{aligned}$$

- sample a degree- $k$  polynomial  $f(X) \in \mathbb{Z}_p^{\leq k}[X]$  with uniformly random coefficients, and set the constant term  $f(0) = sk$
- commit to  $f(X)$  with PC using randomness  $r$ :

$$(\text{com}_{\text{PC}}, \text{aux}_{\text{PC}}) \leftarrow \text{PC.Commit}(f(X); r)$$

- commit to the vector of tuples  $L = \{(i, m_i, \pi_{\text{VC}, i}, f(i))\}_{i \in [n]}$  using a *hiding* Merkle commitment<sup>3</sup>:

$$(\text{com}_{\text{MT}}, \text{aux}_{\text{MT}}) \leftarrow \text{MT.Commit}(L)$$

- output  $\text{com} := (\text{com}_{\text{MT}}, \text{com}_{\text{PC}}, \text{com}_{\text{VC}})$ , and  $\text{aux} := (\text{aux}_{\text{PC}}, \text{aux}_{\text{MT}}, \text{aux}_{\text{VC}})$

---

<sup>3</sup>The hash function will be modeled as a random oracle



- $\text{Open}(\text{sk}, \text{com}, \text{aux}, i) \rightarrow (m_i, \pi) :$ 
  - parse  $\text{com} = (\text{com}_{\text{MT}}, \text{com}_{\text{PC}}, \text{com}_{\text{VC}})$  and  $\text{aux} = (\text{aux}_{\text{PC}}, \text{aux}_{\text{MT}}, \text{aux}_{\text{VC}})$ .
  - compute  $(f(i), \pi_{\text{PC},i}) \leftarrow \text{PC.Open}(\text{com}_{\text{PC}}, \text{aux}_{\text{PC}}, i)$
  - compute  $((i, m_i, \pi_{\text{VC},i}, f(i)), \pi_{\text{MT},i}) \leftarrow \text{MT.Open}(\text{com}_{\text{MT}}, \text{aux}_{\text{MT}}, i)$
  - generate a proof  $\pi_{\text{NIZK}}$  for the statement-witness pair  $((\text{pk}, \text{com}_{\text{PC}}), (\text{sk}, f(X), r))$ .
  - output  $m_i$  and  $\pi = (f(i), \pi_{\text{PC},i}, \pi_{\text{MT},i}, \pi_{\text{VC},i}, \pi_{\text{NIZK}})$
- $\text{Verify}(\text{pk}, \text{com}, i, m_i, \pi) :$ 
  - parse  $\text{com} = (\text{com}_{\text{MT}}, \text{com}_{\text{PC}}, \text{com}_{\text{VC}})$  and  $\pi = (y_i, \pi_{\text{PC},i}, \pi_{\text{MT},i}, \pi_{\text{VC},i}, \pi_{\text{NIZK}})$
  - check that  $\text{PC.Verify}(\text{com}_{\text{PC}}, i, y_i, \pi_{\text{PC},i}) = 1$
  - check that  $\text{MT.Verify}(\text{com}_{\text{MT}}, i, (i, m_i, \pi_{\text{VC},i}, y_i, \pi_{\text{MT},i})) = 1$
  - check that  $\text{VC.Verify}(\text{com}_{\text{VC}}, i, m_i, \pi_{\text{VC},i}) = 1$
  - check that  $\text{NIZK.Verify}((\text{pk}, \text{com}_{\text{PC}}), \pi_{\text{NIZK}}) = 1$
  - output 1 if all checks pass, else output 0

**Remark 3.** Note that, following the definition of non-delegatable commitments (Definition 5),  $\text{Open}$  is part of the protocol to show that the commitment was computed by the “receiver”, i.e., the holder of  $\text{pk}$ . The receiver will not reveal the output of  $\text{Open}$  for more than  $k$  positions, lest it leaks its secret key. Commitments from the above construction (and the concrete one in Section 6.2) can however also be used as “standard” commitments by simply opening the contained vector commitment  $\text{com}_{\text{VC}}$ , and there is no limit on the number of times this can be done.

## 6.2 Concrete Instantiation

We next instantiate the construction with (a variant of the) KZG polynomial commitments. KZG commitments are extractable in the algebraic group model [FKL18] and can be made statistically hiding, as described in Section 4.4, so that opening proofs do not reveal anything else [KT24]. We denote this variant by KZG. Moreover, they have the property that opening proofs for a value  $y$  at position  $x$  can be verified given only  $g^y$ . Assuming the standard key-pair relation  $(\text{pk}, \text{sk}) \in \mathcal{R} \Leftrightarrow \text{pk} = g^{\text{sk}}$ , we can thus replace the NIZK proof by a KZG opening proof at  $x = 0$  and check that it verifies with respect to  $\text{pk}$ . Given that our construction is black-box in the vector commitment scheme, it can be instantiated with a Merkle tree (say), without affecting efficiency.

- $\text{KeyGen}(1^\lambda) :$ 
  - sample  $\text{sk} \leftarrow \mathbb{Z}_p$ .
  - set  $\text{pk}^1 = g^{\text{sk}}$ ,  $\text{pk}^2 = h^{\text{sk}}$ , and  $\text{pk} = (\text{pk}^1, \text{pk}^2)$ .
  - output  $(\text{pk}, \text{sk}) \leftarrow \mathcal{R}$ .
- $\text{Setup}(1^\lambda, 1^n, 1^k) :$ 
  - samples  $\text{crs}_{\text{KZG}} \leftarrow \text{KZG.Setup}(1^\lambda, 1^k)$
  - samples  $\text{crs}_{\text{VC}} \leftarrow \text{VC.Setup}(1^\lambda, 1^n)$
  - outputs  $\text{crs} := (\text{crs}_{\text{KZG}}, \text{crs}_{\text{VC}})$ .
- $\text{Commit}(\text{sk}, m) :$ 
  - commit to  $m \in \mathcal{M}^n$  with VC and compute opening proofs for all points:

$$\begin{aligned}
 (\text{com}_{\text{VC}}, \text{aux}_{\text{VC}}) &\leftarrow \text{VC.Commit}(m) \\
 \{(m_i, \pi_{\text{VC},i}) &\leftarrow \text{VC.Open}(\text{com}_{\text{VC}}, \text{aux}_{\text{VC}}, i)\}_{i \in [n]}
 \end{aligned}$$

- sample a degree- $k$  polynomial  $f(X) \in \mathbb{Z}_p^{\leq k}[X]$  with uniformly random coefficients, and set the constant term to  $f(0) = \text{sk}$
- commit to  $f(X)$  with KZG:  $(\text{com}_{\text{KZG}}, \text{aux}_{\text{KZG}}) \leftarrow \text{KZG.Commit}(f(X))$
- commit to the vector of tuples  $L = \{(i, m_i, \pi_{\text{VC},i}, f(i))\}_{i \in [n]}$  using a *hiding* Merkle commitment:

$$(\text{com}_{\text{MT}}, \text{aux}_{\text{MT}}) \leftarrow \text{MT.Commit}(L)$$

- output  $\text{com} := (\text{com}_{\text{MT}}, \text{com}_{\text{KZG}}, \text{com}_{\text{VC}})$  and  $\text{aux} := (\text{aux}_{\text{KZG}}, \text{aux}_{\text{MT}}, \text{aux}_{\text{VC}})$

- $\text{Open}(\text{sk}, \text{com}, \text{aux}, i) \rightarrow (m_i, \pi)$  :
  - parse  $\text{com} = (\text{com}_{\text{MT}}, \text{com}_{\text{KZG}}, \text{com}_{\text{VC}})$  and  $\text{aux} = (\text{aux}_{\text{KZG}}, \text{aux}_{\text{MT}}, \text{aux}_{\text{VC}})$
  - compute  $(\text{sk}, \pi_{\text{KZG},0}) \leftarrow \text{KZG.Open}(\text{com}_{\text{KZG}}, \text{aux}_{\text{KZG}}, 0)$
  - compute  $(f(i), \pi_{\text{KZG},i}) \leftarrow \text{KZG.Open}(\text{com}_{\text{KZG}}, \text{aux}_{\text{KZG}}, i)$
  - compute  $((i, m_i, \pi_{\text{VC},i}, f(i)), \pi_{\text{MT},i}) \leftarrow \text{MT.Open}(\text{com}_{\text{MT}}, \text{aux}_{\text{MT}}, i)$
  - output  $m_i$  and  $\pi = (f(i), \pi_{\text{KZG},0}, \pi_{\text{KZG},i}, \pi_{\text{MT},i}, \pi_{\text{VC},i})$
- $\text{Verify}(\text{pk}, \text{com}, i, m_i, \pi)$  :
  - check  $e(\text{pk}^1, h) = e(g, \text{pk}^2)$ .
  - parse  $\text{com} = (\text{com}_{\text{MT}}, \text{com}_{\text{KZG}}, \text{com}_{\text{VC}})$  and  $\pi = (f(i), \pi_{\text{KZG},0}, \pi_{\text{KZG},i}, \pi_{\text{MT},i}, \pi_{\text{VC},i})$
  - parse  $\pi_{\text{KZG},0} := (\pi_{\text{KZG},0}^1, \pi_{\text{KZG},0}^2)$ , retrieve  $h^\tau$  and  $h^\eta$  from  $\text{crs}_{\text{KZG}}$ , and check:

$$e(\text{com}_{\text{KZG}}/\text{pk}^1, h) = e(\pi_{\text{KZG},0}^1, h^\tau) \cdot e(\pi_{\text{KZG},0}^2, h^\eta)$$

- check that  $\text{KZG.Verify}(\text{com}_{\text{KZG}}, i, f(i), \pi_{\text{KZG},i}) = 1$
- check that  $\text{MT.Verify}(\text{com}_{\text{MT}}, i, (i, m_i, \pi_{\text{VC},i}, f(i)), \pi_{\text{MT},i}) = 1$
- check that  $\text{VC.Verify}(\text{com}_{\text{VC}}, i, m_i, \pi_{\text{VC},i}) = 1$
- output 1 if all checks pass, else output 0

**Remark 4.** In practice, when the opening algorithm is invoked  $k$  times for the challenge indices, instead of returning a separate opening proof  $\pi_{\text{KZG},x_i}$  for each challenge index  $x_i \in \text{chal}$ , we can return a single batch opening proof for all of the  $k$  indices using the batch opening algorithm of the statistically hiding KZG scheme from Section 4.4:  $\pi_{\text{KZG}} \leftarrow \text{KZG.BatchOpen}(\text{com}_{\text{KZG}}, \text{aux}_{\text{KZG}}, \text{chal})$ . This reduces the number of pairing operations for the verifier from  $O(k)$  to  $O(1)$ .

### 6.3 Security of our Constructions

**Theorem 2.** If VC is a computationally position-binding vector commitment scheme, PC is a hiding and computationally evaluation-binding polynomial commitment scheme, and the NIZK scheme satisfies knowledge soundness and computational zero-knowledge, the construction in Section 6.1 is a secure non-delegatable commitment scheme with  $\mu(k) = (2^{-\lambda/k} - k/n)$ -soundness in the random oracle model.

**Theorem 3.** If VC is a computationally position-binding vector commitment scheme, and the  $(k+1)$ -DLOG assumption holds, then the construction in Section 6.2 is a secure non-delegatable commitment scheme with  $\mu(k) = (2^{-\lambda/k} - k/n)$ -soundness in the random oracle model against algebraic adversaries.

To illustrate the concrete soundness guarantee, we show that our schemes require the receiver to obtain  $\mu(k) = 2^{-\lambda/k} - k/n$  fraction of the messages from a sender to answer the queries (without revealing its secret key). For  $\lambda = 128$ ,  $k = \lambda$ , and  $n \gg k$ ,  $\mu(k) \approx 1/2$ ; whereas for  $k = 1000$  and  $n = 1,000,000$  messages,  $\mu(k) > 93\%$  with only  $k/n = 0.1\%$  of the indices being challenged.

**Correctness.** Correctness of both schemes follows from the correctness of the vector and polynomial commitment schemes and Merkle tree commitments, as well as completeness of the NIZK proof. The formal proof is presented in Appendix A.1.

**Binding and Position-Binding.** Binding and Position-Binding follow from the respective properties of the Merkle and polynomial commitments (i.e., Binding and Evaluation-Binding for polynomial commitments).

**Succinctness.** If the vector and polynomial commitment schemes VC and PC have succinct commitments, opening proofs and verification time, then the generic construction in Section 6.1 satisfies succinctness. If the vector commitment scheme VC has succinct commitments, opening proofs and verification time, then the construction in Section 6.2 satisfies succinctness, as the KZG scheme has succinct commitments, evaluation proofs and verification time.

**Key-Hiding.** We first prove the key-hiding property for the generic construction.

*Proof of Key-Hiding for the Construction in Section 6.1.* We begin by describing the simulator  $\mathcal{S}(\text{pk}, m)$  which works as follows:

1. sample a uniformly random  $k$ -sized subset  $\text{chal} \subseteq [n]$
2. sample  $\text{crs}_{\text{PC}} \leftarrow \text{PC.Setup}(1^\lambda, 1^k)$  and  $\text{crs}_{\text{VC}} \leftarrow \text{VC.Setup}(1^\lambda, 1^n)$
3. commit to  $m$  using VC and compute opening proofs for all indices:

$$\begin{aligned} (\text{com}_{\text{VC}}, \text{aux}_{\text{VC}}) &\leftarrow \text{VC.Commit}(m) \\ \{(m_i, \pi_{\text{VC}, i}) &\leftarrow \text{VC.Open}(\text{com}_{\text{VC}}, \text{aux}_{\text{VC}}, i)\}_{i \in [n]} \end{aligned}$$

4. sample a degree- $k$  polynomial  $f(X) \in \mathbb{Z}_p^{\leq k}[X]$  uniformly at random subject to  $f(0) = 0$
5. compute  $(\text{com}_{\text{PC}}, \text{aux}_{\text{PC}}) \leftarrow \text{PC.Commit}(f(X))$  and the corresponding opening proofs

$$\{(f(i), \pi_{\text{PC}, i}) \leftarrow \text{PC.Open}(\text{com}_{\text{PC}}, \text{aux}_{\text{PC}}, i)\}_{i \in \text{chal}}$$

6. define a vector of quadruples  $L$ , where for  $i \in [n]$ :

$$L_i = \begin{cases} (i, m_i, \pi_{\text{VC}, i}, f(i)) & \text{if } i \in \text{chal}, \\ (0, 0, 0, 0) & \text{otherwise} \end{cases}$$

7. compute a hiding Merkle commitment to  $L$  as  $(\text{com}_{\text{MT}}, \text{aux}_{\text{MT}}) \leftarrow \text{MT.Commit}(L)$  and the corresponding opening proofs

$$\{((i, m_i, \pi_{\text{VC}, i}, f(i)), \pi_{\text{MT}, i}) \leftarrow \text{MT.Open}(\text{com}_{\text{MT}}, \text{aux}_{\text{MT}}, i)\}_{i \in \text{chal}}$$

8. run the NIZK simulator  $\mathcal{S}_{\text{NIZK}}(1^\lambda, (\text{pk}, \text{com}_{\text{PC}}))$  to obtain  $\text{crs}_{\text{NIZK}}$  and  $\pi_{\text{NIZK}}$
9. output  $\text{crs} := (\text{crs}_{\text{PC}}, \text{crs}_{\text{VC}}, \text{crs}_{\text{NIZK}})$ ,  $\text{pk}$ ,  $\text{chal}$ ,  $\text{com} := (\text{com}_{\text{MT}}, \text{com}_{\text{PC}}, \text{com}_{\text{VC}})$  and the proofs  $\{\pi_i = (f(i), \pi_{\text{PC}, i}, \pi_{\text{MT}, i}, \pi_{\text{VC}, i}, \pi_{\text{NIZK}})\}_{i \in \text{chal}}$

To argue that this is a good simulator, we define a sequence of hybrids, starting with faithfully executing the specified algorithms and ending in a hybrid whose output is as produced by the simulator above, without any knowledge of secrets.

**H<sub>0</sub>:** Given  $(\text{pk}, \text{sk})$  and  $m$ :

- sample  $\text{crs} \leftarrow \text{Setup}(1^\lambda, 1^n)$
- compute  $(\text{com}, \text{aux}_{\text{com}}) \leftarrow \text{Commit}(\text{sk}, m)$
- sample a  $k$ -sized subset  $\text{chal} \subseteq [n]$ ,
- compute  $\{(m_i, \pi_i) \leftarrow \text{Open}(\text{sk}, \text{com}, \text{aux}_{\text{com}}, i)\}_{i \in \text{chal}}$
- output  $(\text{crs}, \text{pk}, \text{com}, \text{chal}, m, \{\pi_i\}_{i \in \text{chal}})$

- H<sub>1</sub>:** This hybrid is identical to the previous hybrid, except that  $\pi_{\text{NIZK}}$  is simulated. This also changes the setup algorithm, and we output the  $\text{crs}_{\text{NIZK}}$  at the end, as described in the simulator above. By zero knowledge of the NIZK-AoK, this hybrid is computationally indistinguishable from the previous hybrid.
- H<sub>2</sub>:** In this hybrid, we sample  $\text{chal}$  at the beginning of the protocol instead of after the commitment is created. This hybrid has an identical distribution to the previous hybrid.
- H<sub>3</sub>:** In this hybrid, we replace all unopened leaves of the Merkle tree with  $(0, 0, 0, 0)$  as described in the simulator above. This is now possible because we sampled the challenge before the commitment was created. This hybrid is computationally indistinguishable from the previous hybrid because of the hiding property of the Merkle commitment.
- H<sub>4</sub>:** This is the final hybrid, where we change the polynomial that is committed by the PC by setting  $f(0) = 0$ , instead of  $f(0) = \text{sk}$ . The remaining evaluations at the  $k$  points in  $\text{chal}$  are left unchanged. This hybrid is computationally indistinguishable from the previous hybrid due to the hiding property of the polynomial commitment scheme.

Thus, the scheme in Section 6.1 satisfies key-hiding.  $\square$

We next prove the key-hiding property for the construction instantiated with KZG commitments.

*Proof of Key-Hiding for the Construction in Section 6.2.* We begin by describing the simulator  $\mathcal{S}(\text{pk}, m)$  which works as follows:

- sample a uniformly random  $k$ -sized subset  $\text{chal} \subseteq [n]$
- sample  $\text{crs}_{\text{VC}} \leftarrow \text{VC.Setup}(1^\lambda, 1^n)$
- commit to  $m$  using VC and compute opening proofs for all points:

$$(\text{com}_{\text{VC}}, \text{aux}_{\text{VC}}) \leftarrow \text{VC.Commit}(m)$$

$$\{m_i, \pi_{\text{VC}, i} \leftarrow \text{VC.Open}(\text{com}_{\text{VC}}, \text{aux}_{\text{VC}}, i)\}_{i \in [n]}$$

- sample a degree- $k$  polynomial  $f(X) \in \mathbb{Z}_p^{\leq k}[X]$  uniformly at random subject to  $f(0) = 0$
- to program  $\text{crs}_{\text{KZG}}$ , sample two uniformly random integers  $\tau, \Delta \leftarrow \mathbb{Z}_p$ . Set

$$g^\eta := (\text{pk}^1)^{\frac{1}{\Delta}} \quad h^\eta := (\text{pk}^2)^{\frac{1}{\Delta}}$$

and compute the rest of  $\text{crs}_{\text{KZG}}$  honestly using  $\tau$ .

- sample a uniformly random integer  $r \leftarrow \mathbb{Z}_p$  and compute  $(\text{com}_{\text{PC}}, \text{aux}_{\text{PC}}) \leftarrow \text{PC.Commit}(f(X); r)$  using the randomness  $r$ , along with the corresponding opening proofs:

$$\{(f(i), \pi_{\text{PC}, i}) \leftarrow \text{PC.Open}(\text{com}_{\text{PC}}, \text{aux}_{\text{PC}}, i)\}_{i \in \text{chal}}$$

- define a vector of quadruples  $L$ , where for  $i \in [n]$ :

$$L_i = \begin{cases} (i, m_i, \pi_{\text{VC}, i}, f(i)) & \text{if } i \in \text{chal}, \\ (0, 0, 0, 0) & \text{otherwise} \end{cases}$$

- compute a hiding Merkle commitment to  $L$  as  $(\text{com}_{\text{MT}}, \text{aux}_{\text{MT}}) \leftarrow \text{MT.Commit}(L)$  and the corresponding opening proofs

$$\{((i, m_i, \pi_{\text{VC}, i}, f(i)), \pi_{\text{MT}, i}) \leftarrow \text{MT.Open}(\text{com}_{\text{MT}}, \text{aux}_{\text{MT}}, i)\}_{i \in \text{chal}}$$

- sample a uniformly random integer  $s \leftarrow \mathbb{Z}_p$  and compute  $\pi_{\text{KZG},0}$  as

$$\pi_{\text{KZG},0}^1 = g^{f(\tau)/\tau} \cdot (g^\eta)^s, \quad \pi_{\text{KZG},0}^2 = g^{(r-\Delta)-s\tau}$$

Note that the simulator uses  $r - \Delta$  instead of  $r$  while computing  $\pi_{\text{KZG},0}^2$ , whereas for an honestly computed opening proof at 0, it would have used  $r$ .

- output  $\text{crs} := (\text{crs}_{\text{KZG}}, \text{crs}_{\text{VC}})$ ,  $\text{pk}$ ,  $\text{chal}$ ,  $\text{com} := (\text{com}_{\text{MT}}, \text{com}_{\text{KZG}}, \text{com}_{\text{VC}})$  and the proofs

$$\{\pi_i = (f(i), \pi_{\text{KZG},0}, \pi_{\text{KZG},i}, \pi_{\text{MT},i}, \pi_{\text{VC},i})\}_{i \in \text{chal}}$$

To argue that this is a good simulator, we again define a sequence of hybrids.

- H<sub>0</sub>**: This is the same as **H<sub>0</sub>** from the proof of Key-Hiding for the generic construction in Section 6.1.
- H<sub>1</sub>**: This hybrid is identical to the previous hybrid, except that  $\text{crs}_{\text{KZG}}$  is simulated with  $g^\eta$  and  $h^\eta$  as described above. Since  $\text{sk}/\Delta$  is uniformly distributed over  $\mathbb{Z}_p$ ,  $\text{crs}_{\text{KZG}}$  has the same distribution as the honestly-sampled CRS for the KZG scheme, implying that this hybrid has the same distribution as the previous hybrid.
- H<sub>2</sub>**: This is the same as **H<sub>2</sub>** from the proof of Key-Hiding for the generic construction in Section 6.1.
- H<sub>3</sub>**: This is the same as **H<sub>3</sub>** from the proof of Key-Hiding for the generic construction in Section 6.1.
- H<sub>4</sub>**: This is the final hybrid, where we change the committed polynomial by setting  $f(0) = 0$ , instead of  $f(0) = \text{sk}$ , and compute  $\pi_{\text{KZG},0}$  using  $r - \Delta$  instead of  $r$ , as done by the simulator. Now, for  $\pi_{\text{KZG},0}$ , it holds that

- $\pi_{\text{KZG},0}^1$  has the same marginal distribution in both hybrids **H<sub>3</sub>** and **H<sub>4</sub>**, which is uniform over  $\mathbb{G}_1$  and is independent from all other  $\pi_{\text{KZG},i}^1$  for  $i \neq 0$ .
- Since in **H<sub>4</sub>**,

$$\begin{aligned} e(\text{com}_{\text{KZG}}/\text{pk}^1, h) &= e(g^{f(\tau)+r\eta-\text{sk}}, h) = e(g^{f(\tau)}, h) \cdot e((\text{pk}^1)^{\frac{r}{\Delta}-1}, h) = e(g^{f(\tau)+(r-\Delta)\eta}, h) \\ &= e(g^{f(\tau)+s\eta\tau+(r-\Delta)\eta-s\eta\tau}, h) = e(\pi_{\text{KZG},0}^1, h^\tau) \cdot e(\pi_{\text{KZG},0}^2, h^\eta) \end{aligned}$$

$\pi_{\text{KZG},0}^2$  has the same distribution in both hybrids, which is

$$(\text{com}_{\text{KZG}}/((\pi_{\text{KZG},0}^1)^\tau \cdot \text{pk}^1))^{1/\eta}$$

Therefore,  $\pi_{\text{KZG},0}$  in **H<sub>4</sub>** has the same distribution as the honestly computed opening proof for 0 in **H<sub>3</sub>**. By a similar argument, in both hybrids, for any  $i \in \mathbb{Z}_p$ ,  $i \neq 0$ ,  $\pi_{\text{KZG},i}^1$  has uniform distribution over  $\mathbb{G}_1$  and is independent from all other  $\pi_{\text{KZG},j}^1$ ,  $j \neq i$ , and  $\pi_{\text{KZG},i}^2$  is a deterministic function of  $(\text{com}_{\text{KZG}}, i, g^{f(i)}, \pi_{\text{KZG},i}^1)$ <sup>4</sup>. Moreover, the two collections of the  $k$  ‘ $f(i)$ ’ values revealed in **H<sub>3</sub>** and **H<sub>4</sub>** for the same indices  $i$  as part of the final proof are statistically indistinguishable. Hence, **H<sub>4</sub>** is statistically indistinguishable from **H<sub>3</sub>**.

Thus, the scheme in Section 6.2 satisfies key-hiding. □

**Non-Delegatability.** We begin by providing intuition for why our schemes satisfy non-delegatability with soundness error  $\mu(k) = 2^{-\lambda/k} - k/n$ . For the adversary to have a non-negligible probability of winning the  $\mu(k)$ -soundness game, over the choice of randomness used in sampling  $\text{chal}$ , **R<sub>2</sub>** must successfully provide openings for a “large fraction” of points in  $[n]$ . In particular, if it can only provide openings for  $n - t$  points, then its winning probability is at most  $(1 - t/n)^k$ . For all  $t = \Theta(n)$ , there exists a sufficiently large  $k$  so that

<sup>4</sup>This observation also implies the hiding property for the KZG scheme

this probability is negligible. We then show that, except with negligible probability, each leaf opened by  $\mathbf{R}_2$  must appear in the joint set of queries made by  $\mathbf{S}_1, \dots, \mathbf{S}_\ell$  and  $\mathbf{R}_1$  to the random oracle (Lemma 1).

Next, we argue that  $\mathbf{R}_1$  must have made at least  $n - t - k$  of these queries. If more than  $k$  queries were made by the set  $\mathbf{S}_1, \dots, \mathbf{S}_\ell$ , then some  $\mathbf{S}_j$  must have made a query outside the set  $\text{chal}$  (of size  $k$ ). Together with the openings for  $\text{chal}$ ,  $\mathbf{S}_j$  thus has  $k + 1$  points of the polynomial  $f(X)$  and can reconstruct the secret  $f(0)$ . The extractor can do so too, by identifying the queries corresponding to the Merkle leaves by comparing against the vector commitment openings in each leaf node.

For intuition, we present an abridged version of the proof for the generic construction in Section 6.1. The full proof of non-delegatability for the constructions in Section 6.1 and Section 6.2 is presented in Appendix A.2. We start by showing that, in general, if an adversary is able to successfully produce Merkle opening proofs for a randomly chosen challenge ( $k$ -sized subset), then it must be the case that a large fraction of leaf nodes are queried at the time of creation of the commitment.

**Lemma 1.** *Consider the following game for a PPT adversary  $\mathcal{B} = (\mathcal{B}_1, \mathcal{B}_2)$ :*

- $\mathcal{B}_1(\text{crs}) \rightarrow (\text{com}_{\text{MT}}, \text{st})$
- *Uniformly sample a  $k$ -sized subset  $\text{chal} \subseteq [n]$*
- $\mathcal{B}_2(\text{st}, \text{chal}) \rightarrow \{M_i, \pi_{\text{MT}, i}\}_{i \in \text{chal}}$
- *output 1 if  $\text{MT.Verify}(\text{com}_{\text{MT}}, i, M_i, \pi_{\text{MT}, i}) = 1$  for all  $i \in \text{chal}$ , else output 0*

*For all PPT adversaries  $\mathcal{B} = (\mathcal{B}_1, \mathcal{B}_2)$  that pass the above game with non-negligible probability, there exists a polynomial-time extractor  $\text{Ext}_{\text{MT}}$ , which, given access to  $\text{com}_{\text{MT}}$  and the random oracle queries  $Q_{\mathcal{B}_1}$  made by  $\mathcal{B}_1$ , outputs  $\{M_i, \pi_{\text{MT}, i}\}_{i \in [n]} \leftarrow \text{Ext}_{\text{MT}}(\text{crs}, Q_{\mathcal{B}_1}, \text{com}_{\text{MT}})$  such that*

$$\Pr \left[ \left| \{i \in [n] \mid \text{MT.Verify}(\text{com}_{\text{MT}}, i, M_i, \pi_{\text{MT}, i}) = 1\} \right| \leq (1 - \nu(k))n \right] \leq \text{negl}(\lambda)$$

for  $\nu(k) = 1 - 2^{-\lambda/k}$ .

*Proof.* The extractor follows a “top-down” approach, starting at the root  $\text{com}_{\text{MT}}$  and making its way down to the leaf nodes. If at any point, it encounters an intermediate node that is not in  $Q_{\mathcal{B}_1}$ , then it must be the case that all leaf nodes that are children of this intermediate node cannot be opened. On all paths where it does not encounter such an intermediate node missing from  $Q_{\mathcal{B}_1}$ , the extractor manages to successfully identify the leaf nodes. At the end, it simply outputs all the reachable leaf nodes and their corresponding paths.

All that is left to argue is that the extractor outputs sufficiently many good leaf nodes (and paths). Let  $p$  be the fraction of unreachable leaf nodes. It is sufficient to show that if  $p > \nu(k)$ , then  $\mathcal{B}$  cannot pass the game with non-negligible probability, which leads to a contradiction. The probability that none of the challenge points land on unreachable nodes is at most  $(1 - p)^k$ , which is  $\leq 2^{-\lambda}$ , and thus negligible, for all  $p \geq 1 - 2^{-\lambda/k} = \nu(k)$ .<sup>5</sup>  $\square$

Let  $\mathcal{A} = (\mathbf{S}_1, \dots, \mathbf{S}_\ell, \mathbf{R}_1, \mathbf{R}_2)$  be an adversary that wins the soundness game with non-negligible probability, thus  $\mathbf{R}_2$  returns  $(m'_i, \pi_i = \{y_i, \pi_{\text{PC}, i}, \pi_{\text{MT}, i}, \pi_{\text{VC}, i}, \pi_{\text{NIZK}, i}\}_{i \in \text{chal}})$  with valid  $\pi_i$ , which implies for all  $i \in \text{chal}$ :

1.  $\text{VC.Verify}(\text{com}_{\text{VC}}, i, m_i, \pi_{\text{VC}, i}) = 1$
2.  $\text{PC.Verify}(\text{com}_{\text{PC}}, i, y_i, \pi_{\text{PC}, i}) = 1$
3.  $\text{NIZK.Verify}((\text{pk}, \text{com}_{\text{PC}}), \pi_{\text{NIZK}, i}) = 1$

By knowledge soundness of NIZK, from (all)  $\pi_{\text{NIZK}, i}$  we can extract (with overwhelming probability) a polynomial  $f'(X)$  of degree at most  $k$  and randomness  $r'$  s.t.

<sup>5</sup>As the number of queries  $k$  increase, we can detect progressively smaller fractions of bad leaves with overwhelming probability. The parameter  $\lambda$  determines the error, and  $k$  determines the fraction of messages that can be output at that error rate.

4.  $(\text{com}_{\text{PC}}, \text{aux}_{\text{PC}}) = \text{PC.Commit}(f'(X); r')$  (for some  $\text{aux}_{\text{PC}}$ ) and
5.  $(f'(0), \text{pk}) \in \mathcal{R}$

(By Binding of PC, all  $\pi_{\text{NIZK}, i}$  must yield the same  $f'(X)$ .)

We now adapt Lemma 1 to the soundness game, by letting  $\mathcal{B}_1 := (\mathbf{S}_1, \dots, \mathbf{S}_\ell, \mathbf{R}_1)$  and  $\mathcal{B}_2 := (\mathbf{S}_1, \dots, \mathbf{S}_\ell, \mathbf{R}_2)$  and  $f'(X)$  the polynomial extracted from the NIZK output by  $\mathbf{R}_2$ . If  $\mathcal{A} = (\mathbf{S}_1, \dots, \mathbf{S}_\ell, \mathbf{R}_1, \mathbf{R}_2)$  wins the soundness game with non-negligible probability, that is,  $\mathbf{R}_2$ 's output satisfies the first two bullet points in 6. in Fig. 1, then for a set  $I$  of size at least  $(1 - \nu(k))n$ ,  $\mathcal{B}_1$  must have made random oracle queries of the form

$$\{(i, \hat{m}_i, \hat{\pi}_{\text{VC}, i}, \hat{y}_i)\}_{i \in I},$$

so that the content satisfies the following:

6.  $\hat{m}_i = m_i$
7.  $\text{VC.Verify}(\text{com}_{\text{VC}}, i, \hat{m}_i, \hat{\pi}_{\text{VC}, i}) = 1$

We moreover argue that for these queries:

8.  $\hat{y}_i = f'(i)$

Assume  $|I| < (1 - \nu(k))n$ . Then, as argued in Lemma 1, with non-negligible probability,  $\text{chal}$  will contain an index  $i^*$  not in  $I$ .

Assume 8. does not hold for  $i^*$ . Then, running  $\text{PC.Open}$  using  $\text{aux}_{\text{PC}}$  from 4. produces a proof  $\pi'_{\text{PC}, i^*}$  for  $f'(i^*)$ , while from 2. we have that  $\pi_{\text{PC}, i^*}$  is a valid proof for  $\hat{y}_{i^*} \neq f'(i^*)$ . Together this breaks PC evaluation binding.

We are now ready to construct the extractors  $\text{Ext}^{\mathbf{S}_j}$  and  $\text{Ext}^{\mathbf{R}_1}$ .

$\text{Ext}^{\mathbf{R}_1}(\text{pk}, \text{com}, (\mathbf{R}_1, (\mathbf{S}_1, \dots, \mathbf{S}_\ell), \cdot)) \rightarrow m''$ : Unlike in the proof of Lemma 1, an extractor cannot follow a “top-down” approach to arrive at the leaf nodes, which identifies each leaf query. Indeed,  $\mathbf{R}_1$  can for example delegate computation of the intermediate nodes in the Merkle tree without compromising the privacy of its secret key, as in a hiding Merkle commitment, the leaves are committed with randomness. However, when leaf nodes are contained in the random oracle queries made by  $\mathbf{R}_1$ , the extractor can identify them efficiently by relying on the VC proofs contained in the leaf nodes. More formally,  $\text{Ext}^{\mathbf{R}_1}$  does the following:

- Maintain a list  $\mathcal{L}$  of tuples from  $[n] \times \mathcal{M}$  containing the entries of  $m$  that have been recovered so far.
- Parse  $\text{com} = (\text{com}_{\text{MT}}, \text{com}_{\text{PC}}, \text{com}_{\text{VC}})$ .
- Let  $Q_R$  denote the random oracle queries made by  $\mathbf{R}_1$ . Try parsing each query as  $(i, \bar{m}_i, \bar{\pi}_{\text{VC}, i}, \bar{y}_i)$ , and if  $\text{VC.Verify}(\text{com}_{\text{VC}}, i, \bar{m}_i, \bar{\pi}_{\text{VC}, i}) = 1$ , add it to the list  $\mathcal{L} \leftarrow \mathcal{L} \cup \{(i, \bar{m}_i)\}$ . If some  $(i, \bar{m}'_i)$  with  $\bar{m}'_i \neq \bar{m}_i$  already exists in the list, abort.
- Define  $m''$  where for  $i \in [n]$ 

$$m''_i = \begin{cases} \bar{m}_i & \text{if } (i, \bar{m}_i) \in \mathcal{L}, \\ \perp & \text{otherwise} \end{cases}$$
- Output  $m''$ .

By the Position-Binding of VC, it is infeasible to produce conflicting openings for  $\text{com}_{\text{VC}}$  at the same position  $i$ . Thus,  $\text{Ext}^{\mathbf{R}_1}$  only aborts with negligible probability and moreover (due to the proofs  $\hat{\pi}_{\text{VC}, i}$  (cf. 7) and  $\bar{\pi}_{\text{VC}, i}$ ):

9.  $\hat{m}_i = m''_i$  (whenever both of them are defined)

and by 6.:

10.  $m_i = m''_i$  (whenever  $m''_i \neq \perp$ )



If  $|\{i \in [n] \mid m_i = m_i''\}| > (1 - \mu(k))n$  with  $\mu(k) = 1 - \nu(k) - k/n$ , then  $\mathcal{A}$  loses the soundness game. We next show that if this is not the case, then one of  $\{\text{Ext}^{\mathbf{S}_j}\}_{j \in [\ell]}$  can recover the secret key.

$\text{Ext}^{\mathbf{S}_j}(\text{pk}, \text{com}, \langle \mathbf{S}_j, \mathbf{R}_1 \rangle, \langle \mathbf{S}_j, \mathbf{R}_2, \{(m'_i, \pi_i)\}_{i \in \text{chal}} \rangle) \rightarrow \text{sk}$ : This extractor monitors the random oracle queries for valid leaf nodes by  $\mathbf{S}_j$ . If one of them is outside of  $\text{chal}$ , then using the collected  $k + 1$  points  $y_i$  output by  $\mathbf{R}_2$ , the interpolating polynomial reveals the secret key.

- Maintain a list  $\mathcal{L}$  of tuples from  $[n] \times \mathcal{M} \times \mathbb{Z}_p$  containing the set of evaluations that have been recovered.
- Parse  $\text{com} = (\text{com}_{\text{MT}}, \text{com}_{\text{PC}}, \text{com}_{\text{VC}})$ .
- For  $i \in \text{chal}$ , parse  $\mathbf{R}_2$ 's output  $m'_i$  and  $\pi_i = (y_i, \pi_{\text{PC},i}, \pi_{\text{MT},i}, \pi_{\text{VC},i}, \pi_{\text{NIZK},i})$ .
- Let  $Q_S$  denote the random oracle queries made by  $\mathbf{S}_j$ . Try parsing each query as  $(i, \bar{m}_i, \bar{\pi}_{\text{VC},i}, \bar{y}_i)$ , and if  $i \notin \text{chal}$  and  $\text{VC.Verify}(\text{com}_{\text{VC}}, i, \bar{m}_i, \bar{\pi}_{\text{VC},i}) = 1$ , add it to the list  $\mathcal{L} \leftarrow \mathcal{L} \cup \{(i, \bar{m}_i, \bar{y}_i)\}$ .
- If  $\mathcal{L} = \emptyset$ , abort.
- Else for all entries  $(i, \bar{m}_i, \bar{y}_i) \in \mathcal{L}$ : Interpolate a polynomial  $f_i(X)$  through the point  $(i, \bar{y}_i)$  as well as  $\{(i, y_i)\}_{i \in \text{chal}}$ . If  $(f_i(0), \text{pk}) \in \mathcal{R}$  then stop and output  $f_i(0)$ .
- Else, abort.

Clearly,  $\text{Ext}^{\mathbf{S}_j}$  runs in polynomial time. Moreover, by 6. and 8.,  $(\mathbf{S}_1, \dots, \mathbf{S}_\ell)$  and  $\mathbf{R}_1$  made random oracle queries  $\{(i, \hat{m}_i, \hat{\pi}_{\text{VC},i}, \hat{y}_i)\}_{i \in I}$  for  $|I| \geq (1 - \nu(k))n$  with

11.  $\hat{m}_i = m_i$  and
12.  $\hat{y}_i = f'(i)$

If  $|\{i \in [n] \mid m_i = m_i''\}| \leq (1 - \nu(k))n - k$ , then one of the queries  $i^*$  in  $I$  but outside of  $\text{chal}$  must have been made by some  $\mathbf{S}_j$ , and thus  $(i^*, \hat{m}_{i^*}, \hat{y}_{i^*})$  will be in its list  $\mathcal{L}$ , implying that  $f_{i^*}(X) = f'(X)$  and it will recover  $f'(0)$ , which is a secret key for  $\text{pk}$ . Hence, if  $\text{Ext}^{\mathbf{R}_1}$  outputs at most  $(1 - \nu(k) - k/n)n$  points, a value  $\text{sk}$  such that  $(\text{sk}, \text{pk}) \in \mathbf{R}$  can be extracted from  $\mathbf{S}_j$ 's random oracle queries except with negligible probability. This implies that  $\mathcal{A}$  cannot pass the soundness game except with negligible probability (Fig. 1) and concludes the proof showing that the construction in Section 6.1 is a secure non-delegatable commitment scheme.

**Remark 5.** To ensure that a polynomial-time extractor  $\text{Ext}^{\mathbf{S}_j}$  can identify the polynomial  $f'(X)$ , it is crucial that the degree of the polynomial is at most a constant  $k$  larger than the number of opened leaves.

**Acknowledgments.** We thank Joachim Neu and Paul Resnick for several insightful discussions on this project. This work was performed in part during a research internship of the third author and a research visit of the first author at *a16z crypto*. The first author is supported by the Vienna Science and Technology Fund (WWTF) [10.47379/VRG18002] and by the Austrian Science Fund (FWF) [10.55776/F8515-N].

## References

- [AADG25] Prabhanjan Ananth, Gilad Asharov, Hila Dahari, and Vipul Goyal. Towards accountability in CRS generation. *J. Cryptol.*, 38(4):29, 2025. 6
- [ACH11] Gilad Asharov, Ran Canetti, and Carmit Hazay. Towards a game theoretic view of secure computation. In *EUROCRYPT*, volume 6632 of *Lecture Notes in Computer Science*, pages 426–445. Springer, 2011. 6

- [ADGH06] Ittai Abraham, Danny Dolev, Rica Gonen, and Joseph Y. Halpern. Distributed computing meets game theory: robust mechanisms for rational secret sharing and multiparty computation. In *PODC*, pages 53–62. ACM, 2006. 6
- [AL10] Yonatan Aumann and Yehuda Lindell. Security against covert adversaries: Efficient protocols for realistic adversaries. *J. Cryptol.*, 23(2):281–343, 2010. 6
- [Anz25] Anza / Agave Documentation. Turbine block propagation. <https://docs.anza.xyz/consensus/turbine-block-propagation>, 2025. Accessed: 2025-10-02. 3
- [BFL20] Balthazar Bauer, Georg Fuchsbauer, and Julian Loss. A classification of computational assumptions in the algebraic group model. In *CRYPTO (2)*, volume 12171 of *Lecture Notes in Computer Science*, pages 121–151. Springer, 2020. 30
- [BG17] Vitalik Buterin and Virgil Griffith. Casper the friendly finality gadget. *CoRR*, abs/1710.09437, 2017. 6
- [BGM<sup>+</sup>18] Christian Badertscher, Juan A. Garay, Ueli Maurer, Daniel Tschudi, and Vassilis Zikas. But why does it work? A rational protocol design treatment of bitcoin. In *EUROCRYPT (2)*, volume 10821 of *Lecture Notes in Computer Science*, pages 34–65. Springer, 2018. 6
- [BK22] Dan Boneh and Chelsea Komlo. Threshold signatures with private accountability. In *CRYPTO (4)*, volume 13510 of *Lecture Notes in Computer Science*, pages 551–581. Springer, 2022. 6
- [BKN17] Dan Boneh, Sam Kim, and Valeria Nikolaenko. Lattice-based DAPS and generalizations: Self-enforcement in signature schemes. In *ACNS*, volume 10355 of *Lecture Notes in Computer Science*, pages 457–477. Springer, 2017. 6
- [BLR24] Eric Budish, Andrew Lewis-Pye, and Tim Roughgarden. The economic limits of permissionless consensus. In *EC*, pages 704–731. ACM, 2024. 6
- [BPR24a] Dan Boneh, Aditi Partap, and Lior Rotem. Accountability for misbehavior in threshold decryption via threshold traitor tracing. In *CRYPTO (7)*, volume 14926 of *Lecture Notes in Computer Science*, pages 317–351. Springer, 2024. 6
- [BPR24b] Dan Boneh, Aditi Partap, and Lior Rotem. Proactive refresh for accountable threshold signatures. In *FC (2)*, volume 14745 of *Lecture Notes in Computer Science*, pages 140–159. Springer, 2024. 6
- [BPR24c] Dan Boneh, Aditi Partap, and Lior Rotem. Traceable secret sharing: Strong security and efficient constructions. In *CRYPTO (5)*, volume 14924 of *Lecture Notes in Computer Science*, pages 221–256. Springer, 2024. 6
- [BPR25] Dan Boneh, Aditi Partap, and Lior Rotem. Traceable verifiable random functions. In *CRYPTO (2)*, volume 16001 of *Lecture Notes in Computer Science*, pages 203–234. Springer, 2025. 6
- [BPS17] Mihir Bellare, Bertram Poettering, and Douglas Stebila. Deterring certificate subversion: Efficient double-authentication-preventing signatures. In *Public Key Cryptography (2)*, volume 10175 of *Lecture Notes in Computer Science*, pages 121–151. Springer, 2017. 6
- [BYK21] Osman Biçer, Burcu Yildiz, and Alptekin Küpçü. m-stability: Threshold security meets transferable utility. In *CCSW*, pages 73–82. ACM, 2021. 6
- [CFN94] Benny Chor, Amos Fiat, and Moni Naor. Tracing traitors. In *CRYPTO*, volume 839 of *Lecture Notes in Computer Science*, pages 257–270. Springer, 1994. 6
- [CFS22] Dario Catalano, Georg Fuchsbauer, and Azam Soleimanian. Double-authentication-preventing signatures in the standard model. *J. Comput. Secur.*, 30(1):3–38, 2022. 6

- [DFK<sup>+</sup>25] Stefan Dziembowski, Sebastian Faust, Paweł Kędzior, Marcin Mielniczuk, Susil Kumar Mohanty, and Krzysztof Pietrzak. Beholder signatures. Cryptology ePrint Archive, Paper 2025/1900, 2025. [7](#)
- [DFL23] Stefan Dziembowski, Sebastian Faust, and Tomasz Lizurej. Individual cryptography. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part II*, volume 14082 of *LNCS*, pages 547–579. Springer, Cham, August 2023. [2](#), [7](#)
- [DHR00] Yevgeniy Dodis, Shai Halevi, and Tal Rabin. A cryptographic solution to a game theoretic problem. In Mihir Bellare, editor, *CRYPTO 2000*, volume 1880 of *LNCS*, pages 112–130. Springer, Berlin, Heidelberg, August 2000. [6](#)
- [FKL18] Georg Fuchsbauer, Eike Kiltz, and Julian Loss. The algebraic group model and its applications. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part II*, volume 10992 of *LNCS*, pages 33–62. Springer, Cham, August 2018. [4](#), [11](#), [15](#)
- [FKN10] Georg Fuchsbauer, Jonathan Katz, and David Naccache. Efficient rational secret sharing in standard communication networks. In *TCC*, volume 5978 of *Lecture Notes in Computer Science*, pages 419–436. Springer, 2010. [6](#)
- [Gen09] Craig Gentry. Fully homomorphic encryption using ideal lattices. In *STOC*, pages 169–178. ACM, 2009. [4](#)
- [GK06] S. Dov Gordon and Jonathan Katz. Rational secret sharing, revisited. In *SCN*, volume 4116 of *Lecture Notes in Computer Science*, pages 229–241. Springer, 2006. [6](#)
- [GK12] Adam Groce and Jonathan Katz. Fair computation with rational players. In *EUROCRYPT*, volume 7237 of *Lecture Notes in Computer Science*, pages 81–98. Springer, 2012. [6](#)
- [GKM<sup>+</sup>13] Juan A. Garay, Jonathan Katz, Ueli Maurer, Björn Tackmann, and Vassilis Zikas. Rational protocol design: Cryptography against incentive-driven adversaries. In *54th FOCS*, pages 648–657. IEEE Computer Society Press, October 2013. [6](#)
- [GKS22] Chaya Ganesh, Bhavana Kanukurthi, and Girisha Shankar. Secure auctions in the presence of rational adversaries. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022*, pages 1173–1186. ACM Press, November 2022. [6](#)
- [GKTZ15] Juan A. Garay, Jonathan Katz, Björn Tackmann, and Vassilis Zikas. How fair is your protocol?: A utility-based approach to protocol optimality. In *PODC*, pages 281–290. ACM, 2015. [6](#)
- [GLR13] Ronen Gradwohl, Noam Livne, and Alon Rosen. Sequential rationality in cryptographic protocols. *ACM Trans. Economics and Comput.*, 1(1):2:1–2:38, 2013. [6](#)
- [GSS21] Vipul Goyal, Yifan Song, and Akshayaram Srinivasan. Traceable secret sharing and applications. In *CRYPTO (3)*, volume 12827 of *Lecture Notes in Computer Science*, pages 718–747. Springer, 2021. [6](#)
- [Hal11] Joseph Y. Halpern. Beyond nash equilibrium: Solution concepts for the 21st century. In *GameSec*, volume 7037 of *Lecture Notes in Computer Science*, pages 1–3. Springer, 2011. [6](#)
- [HT04] Joseph Y. Halpern and Vanessa Teague. Rational secret sharing and multiparty computation: extended abstract. In *STOC*, pages 623–632. ACM, 2004. [6](#)
- [KBD<sup>+</sup>24] Mahimna Kelkar, Kushal Babel, Philip Daian, James Austgen, Vitalik Buterin, and Ari Juels. Complete knowledge: Preventing encumbrance of cryptographic secrets. In Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie, editors, *ACM CCS 2024*, pages 2415–2429. ACM Press, October 2024. [2](#)

- [KGP<sup>+</sup>25] Mahimna Kelkar, Aadityan Ganesh, Aditi Partap, Joseph Bonneau, and S. Matthew Weinberg. Breaking omertà: On threshold cryptography, smart collusion, and whistleblowing. *Cryptology ePrint Archive*, Paper 2025/1582, 2025. 6
- [KN08] Gillat Kol and Moni Naor. Cryptography and game theory: Designing protocols for exchanging information. In *TCC*, volume 4948 of *Lecture Notes in Computer Science*, pages 320–339. Springer, 2008. 6
- [KT24] Tohru Kohrita and Patrick Towa. Zeromorph: Zero-knowledge multilinear-evaluation proofs from homomorphic univariate commitments. *J. Cryptol.*, 37(4):38, 2024. 6, 10, 11, 15
- [KZG10] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. Constant-size commitments to polynomials and their applications. In Masayuki Abe, editor, *ASIACRYPT 2010*, volume 6477 of *LNCS*, pages 177–194. Springer, Berlin, Heidelberg, December 2010. 6, 10
- [Mau05] Ueli M. Maurer. Abstract models of computation in cryptography (invited paper). In Nigel P. Smart, editor, *10th IMA International Conference on Cryptography and Coding*, volume 3796 of *LNCS*, pages 1–12. Springer, Berlin, Heidelberg, December 2005. 4
- [PH10] Rafael Pass and Joe Halpern. Game theory with costly computation: formulation and application to protocol security. In *BQGT*, page 89:1. ACM, 2010. 6
- [PS14] Bertram Poettering and Douglas Stebila. Double-authentication-preventing signatures. In *ESORICS (1)*, volume 8712 of *Lecture Notes in Computer Science*, pages 436–453. Springer, 2014. 6
- [RKS15] Tim Ruffing, Aniket Kate, and Dominique Schröder. Liar, liar, coins on fire!: Penalizing equivocation by loss of bitcoins. In *CCS*, pages 219–230. ACM, 2015. 6
- [SCW23] Elaine Shi, Hao Chung, and Ke Wu. What can cryptography do for decentralized mechanism design? In Yael Tauman Kalai, editor, *ITCS 2023*, volume 251, pages 97:1–97:22. LIPIcs, January 2023. 6
- [Sho97] Victor Shoup. Lower bounds for discrete logarithms and related problems. In Walter Fumy, editor, *EUROCRYPT’97*, volume 1233 of *LNCS*, pages 256–266. Springer, Berlin, Heidelberg, May 1997. 4
- [Sol24] Solana Labs. Solana validator requirements. <https://docs.solanalabs.com/operations/requirements>, 2024. Accessed: 2025-10-02. 2
- [SWN<sup>+</sup>21] Peiyao Sheng, Gerui Wang, Kartik Nayak, Sreeram Kannan, and Pramod Viswanath. BFT protocol forensics. In *CCS*, pages 1722–1743. ACM, 2021. 6
- [ZGK<sup>+</sup>17] Yupeng Zhang, Daniel Genkin, Jonathan Katz, Dimitrios Papadopoulos, and Charalampos Papamanthou. A zero-knowledge version of vsql. *IACR Cryptol. ePrint Arch.*, page 1146, 2017. 10
- [Zha25] Mark Zhandry. Optimal traitor tracing from pairings. In *EUROCRYPT (3)*, volume 15603 of *Lecture Notes in Computer Science*, pages 305–335. Springer, 2025. 6

## A Appendix

### A.1 Proof of Correctness

**Theorem 4.** *If the polynomial and vector commitment schemes PC, MT and VC satisfy correctness, both constructions in Section 6.1 and Section 6.2 satisfy correctness, where PC is the KZG scheme from Section 4.4 in the case of the construction in Section 6.2*

*Proof.* By the correctness of PC, MT, and VC, for any commitment  $\text{com}$  and proof  $\pi$  output by an honest receiver such that  $\text{com} := (\text{com}_{\text{MT}}, \text{com}_{\text{PC}}, \text{com}_{\text{VC}})$ , and  $\pi := (f(i), f(x), \pi_{\text{PC},i}, \pi_{\text{PC},x}, \pi_{\text{MT},i}, \pi_{\text{VC},i}, \pi_{\text{NIZK}})$ , the following holds with probability 1:

$$\begin{aligned} \text{NIZK.Verify}(\text{pk}, \text{com}_{\text{PC}}, \pi_{\text{NIZK}}) &= 1 \text{ for the generic construction,} \\ \text{PC.Verify}(\text{com}_{\text{PC}}, i, f(i), \pi_{\text{PC},i}) &= 1 \text{ for } i \in [k], \\ \text{MT.Verify}(\text{com}_{\text{MT}}, i, (i, m_i, \pi_{\text{VC},i}, f(i)), \pi_{\text{MT},i}) &= 1, \\ \text{VC.Verify}(\text{com}_{\text{VC}}, i, m_i, \pi_{\text{VC},i}) &= 1 \\ e(\text{com}_{\text{KZG}}/\text{pk}^1, h) &= e(\pi_{\text{KZG}}^1, h^\tau) \cdot e(\pi_{\text{KZG}}^2, h^\eta) \text{ for the construction with KZG.} \end{aligned}$$

Therefore, both constructions satisfy correctness.  $\square$

## A.2 Proof of Non-delegatability

**Theorem 2.** *If VC is a computationally position-binding vector commitment scheme, PC is a computationally evaluation-binding polynomial commitment scheme, and the NIZK scheme satisfies knowledge soundness, the construction in Section 6.1 satisfies non-delegatability with  $\mu(k) = (2^{-\lambda/k} - k/n)$ -soundness in the random oracle model.*

*Proof.* Consider a PPT adversary  $\mathcal{A} = (\mathbf{S}_1, \dots, \mathbf{S}_\ell, \mathbf{R}_1, \mathbf{R}_2)$  such that for the messages  $m$ , commitment  $\text{com}$  and proof  $\pi$  output by  $\mathcal{A}$ , it holds that  $m_i = m'_i$ , and  $\text{Verify}(\text{pk}, \text{com}, i, m_i, \pi_i) = 1$  for all  $i \in \text{chal}$ . Let  $\text{com} = (\text{com}_{\text{MT}}, \text{com}_{\text{PC}}, \text{com}_{\text{VC}})$  and  $\pi_i = (f(i), \pi_{\text{PC},i}, \pi_{\text{MT},i}, \pi_{\text{VC},i}, \pi_{\text{NIZK},i})$ .

By the knowledge soundness of NIZK, a polynomial  $f'_i(X)$  along with its randomness  $r'_i$ , can be extracted from each  $\pi_{\text{NIZK},i}$  using the expected-polynomial-time extractor for the following relation:

$$\mathcal{R}_{\text{NIZK}} = \left\{ (\text{pk}, \text{com}_{\text{PC}}; \text{sk}, f(X), r) \left| \begin{array}{l} (\text{pk}, \text{sk}) \in \mathcal{R} \\ f(X) \in \mathbb{Z}_p^{\leq k}[X] \\ \text{com}_{\text{PC}} = \text{PC.Commit}(f(X); r) \\ f(0) = \text{sk} \end{array} \right. \right\}$$

Let  $f'_i(X)$  and  $r'_i$  denote the output of this extractor, and note that as  $\pi_{\text{NIZK},i}$  verifies,  $f'_i(X) \in \mathbb{Z}_p^{\leq k}[X]$ ,  $\text{com}_{\text{PC}} = \text{PC.Commit}(f'_i(X); r'_i)$ , and  $(f'_i(X), \text{pk}) \in \mathcal{R}$ , except with negligible probability  $\epsilon_{\text{NIZK}}$ . By the binding property of the PC scheme, all of the polynomials  $f'_i(X)$  are equal, i.e.,  $f'_i(X) = f'(X)$  for all  $i \in \text{chal}$ .

We next prove the following lemma, which follows from similar arguments to Lemma 1. Recall that  $m$  denotes the sequence of messages output by  $(\mathbf{S}_1, \dots, \mathbf{S}_\ell)$ .

**Lemma 2.** *Let  $m \in \mathcal{M}^n$  be a fixed sequence, and consider the following game for a PPT adversary  $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ :*

- $\mathcal{A}_1(\text{crs}) \rightarrow (\text{pk}, \text{com}_{\text{MT}}, \text{com}_{\text{VC}}, \text{com}_{\text{PC}}, \text{aux})$
- *Uniformly sample a  $k$ -sized subset  $\text{chal} \subseteq [n]$*
- $\mathcal{A}_2(\text{chal}, \text{aux}_{\text{MT}}) \rightarrow \{(i, m'_i, \pi'_{\text{VC},i}, y'_i), \pi'_{\text{MT},i}, \pi'_{\text{PC},i}, \pi'_{\text{NIZK},i}\}_{i \in \text{chal}}$
- *output 1 if*
  - $m'_i = m_i$ ,
  - $\text{MT.Verify}(\text{com}_{\text{MT}}, i, (i, m'_i, \pi'_{\text{VC},i}, y'_i), \pi'_{\text{MT},i}) = 1$ ,
  - $\text{VC.Verify}(\text{com}_{\text{VC}}, i, m'_i, \pi'_{\text{VC},i}) = 1$ ,
  - $\text{PC.Verify}(\text{com}_{\text{PC}}, i, y'_i, \pi'_{\text{PC},i}) = 1$ .
  - $\text{NIZK.Verify}((\text{pk}, \text{com}_{\text{PC}}), \pi'_{\text{NIZK},i}) = 1$

*Else, output 0*

For all PPT adversaries  $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$  that pass the above game with non-negligible probability, there exists a polynomial-time extractor  $\text{Ext}_{\text{MT}}$ , which, given access to  $\text{com}_{\text{MT}}$  and the random oracle queries  $Q_{\mathcal{A}_1}$  made by  $\mathcal{A}_1$ , outputs  $\{(i, \hat{m}_i, \hat{\pi}_{\text{VC},i}, \hat{y}_i), \hat{\pi}_{\text{MT},i}\}_{i \in [n]} \leftarrow \text{Ext}_{\text{MT}}(\text{crs}, Q_{\mathcal{A}_1}, \text{com}_{\text{MT}})$  such that for over  $(1 - \nu(k))n$  indices, where  $\nu(k) = 1 - 2^{-\lambda/k}$ , the following holds except with probability  $\text{negl}(\lambda)$  for a degree  $(\leq k)$ -polynomial such that  $(f'(0), \text{pk}) \in \mathcal{R}$ :

1.  $\text{MT.Verify}(\text{com}_{\text{MT}}, i, (i, \hat{m}_i, \hat{\pi}_{\text{VC},i}, \hat{y}_i), \hat{\pi}_{\text{MT},i}) = 1$ ,
2.  $\hat{m}_i = m_i$ ,
3.  $\text{VC.Verify}(\text{com}_{\text{VC}}, i, \hat{m}_i, \hat{\pi}_{\text{VC},i}) = 1$ ,
4.  $\hat{y}_i = f'(i)$ .

*Proof.* The extractor follows a “top-down” approach, starting at the root  $\text{com}_{\text{MT}}$  and making its way down to the leaf nodes. If at any point, it encounters an intermediate node that is not in  $Q_{\mathcal{A}_1}$ , then it must be the case that all leaf nodes that are children of this intermediate node cannot be opened. On all paths where it does not encounter such an intermediate node missing from  $Q_{\mathcal{A}_1}$ , the extractor manages to successfully identify the leaf nodes. At the end, it simply outputs all the reachable leaf nodes and their corresponding paths. For the non-reachable nodes and paths, it outputs  $\perp$ .

We next argue that the extractor outputs over  $(1 - \nu(k))n$  leaf nodes (and Merkle opening proofs) such that for each of these leaf nodes  $(i, \hat{m}_i, \hat{\pi}_{\text{VC},i}, \hat{y}_i)$  and the corresponding opening proofs  $\hat{\pi}_{\text{MT},i}$ , 1.–4. from above hold.

Consider an adversary  $\mathcal{A}$  that passes the game above. Let  $f'_i(X)$  and  $r'_i$  denote the output of the expected-polynomial-time extractor run on  $\pi_{\text{NIZK},i}$  for the relation  $\mathcal{R}_{\text{NIZK}}$ . Note that as  $\pi_{\text{NIZK},i}$  verifies,  $f'_i(X) \in \mathbb{Z}_p^{\leq k}[X]$ ,  $\text{comp}_{\text{PC}} = \text{PC.Commit}(f'_i(X); r'_i)$ , and  $(f'_i(0), \text{pk}) \in \mathcal{R}$ , except with negligible probability  $\epsilon_{\text{NIZK}}$ . By the binding property of the PC scheme, all of the polynomials  $f'_i(X)$  and randomness  $r'_i$  are equal, i.e.,  $f'_i(X) = f'(X)$ ,  $r'_i = r$  for all  $i \in \text{chal}$ .

Suppose for  $\ell_1 \geq \nu(k)n$  indices  $(a_i)_{i \in [\ell_1]} \subseteq [n]$ , at least one of the items above is not true. Then, either  $\text{chal} \cap (a_i)_{i \in [\ell_1]} = \emptyset$ , which happens with probability at most  $(1 - \nu(k))^k = 2^{-\lambda}$ , or there exists an index  $i^* \in \text{chal} \cap (a_i)_{i \in [\ell_1]}$ , which implies the union of the following events:

**Event 1:**  $\text{MT.Verify}(\text{com}_{\text{MT}}, i^*, (i, \hat{m}_{i^*}, \hat{\pi}_{\text{VC},i^*}, \hat{y}_{i^*}), \hat{\pi}_{\text{MT},i^*}) \neq 1$ . This implies  $i^*$  is an unreachable leaf node, which is a contradiction, since  $\mathcal{A}_2$  outputs a leaf  $(i^*, m'_{i^*}, \pi'_{\text{VC},i^*}, y'_{i^*})$  and an opening proof  $\pi'_{\text{MT},i^*}$  such that  $\text{MT.Verify}(\text{com}_{\text{MT}}, i^*, (i, m'_{i^*}, \pi'_{\text{VC},i^*}, y'_{i^*}), \pi'_{\text{MT},i^*}) = 1$ . Therefore, this event cannot happen.

**Event 2:**  $\hat{m}_{i^*} \neq m_{i^*}$ . Then, as  $\mathcal{A}$  passes the game,  $m'_{i^*} = m_{i^*}$ , and

- $\text{MT.Verify}(\text{com}_{\text{MT}}, i^*, (i^*, m_{i^*}, \pi'_{\text{VC},i^*}, y'_{i^*}), \pi'_{\text{MT},i^*}) = 1$ , and

However, in this case,  $(i^*, m_{i^*}, \pi'_{\text{VC},i^*}, y'_{i^*}) \neq (i^*, \hat{m}_{i^*}, \hat{\pi}_{\text{VC},i^*}, \hat{y}_{i^*})$  both verify with respect to  $\text{com}_{\text{MT}}$ , which by the position-binding property of Merkle commitments, happens with probability at most  $\epsilon_{\text{MT}} = \text{negl}(\lambda)$ .

**Event 3.**  $\hat{m}_{i^*} = m_{i^*}$  and  $\text{VC.Verify}(\text{com}_{\text{VC}}, i, \hat{m}_i, \hat{\pi}_{\text{VC},i}) \neq 1$ . Then, as  $\mathcal{A}$  passes the game,

- $\text{MT.Verify}(\text{com}_{\text{MT}}, i^*, (i^*, m_{i^*}, \pi'_{\text{VC},i^*}, y'_{i^*}), \pi'_{\text{MT},i^*}) = 1$ , and
- $\text{VC.Verify}(\text{com}_{\text{VC}}, i^*, m_{i^*}, \pi'_{\text{VC},i^*}) = 1$ .

Now, by the fact that  $\text{VC.Verify}(\text{com}_{\text{VC}}, i^*, \hat{m}_{i^*}, \hat{\pi}_{\text{VC},i^*}) \neq 1$ , and  $\hat{m}_{i^*} = m_{i^*}$ , it must be that  $\hat{\pi}_{\text{VC},i^*} \neq \pi'_{\text{VC},i^*}$ . However, in this case,  $(i^*, m_{i^*}, \pi'_{\text{VC},i^*}, y'_{i^*}) \neq (i^*, m_{i^*}, \hat{\pi}_{\text{VC},i^*}, \hat{y}_{i^*})$  both verify with respect to  $\text{com}_{\text{MT}}$ , which by the position-binding property of Merkle commitments, happens with probability at most  $\epsilon_{\text{MT}} = \text{negl}(\lambda)$ .

**Event 4.**  $\hat{y}_{i^*} \neq f'(i^*)$ . Then, as  $\mathcal{A}$  passes the game,

- $\text{MT.Verify}(\text{com}_{\text{MT}}, i^*, (i^*, m'_{i^*}, \pi'_{\text{VC}, i^*}, y'_{i^*}), \pi'_{\text{MT}, i^*}) = 1$ ,
- $\text{VC.Verify}(\text{com}_{\text{VC}}, i^*, m'_{i^*}, \pi'_{\text{VC}, i^*}) = 1$ ,
- $\text{PC.Verify}(\text{com}_{\text{PC}}, i^*, y'_{i^*}, \pi'_{\text{PC}, i^*}) = 1$ .

Now, consider the following two cases:

- $\hat{y}_{i^*} \neq y'_{i^*}$ : In this case,  $(i^*, m'_{i^*}, \pi'_{\text{VC}, i^*}, y'_{i^*}) \neq (i^*, \hat{m}_{i^*}, \hat{\pi}_{\text{VC}, i^*}, \hat{y}_{i^*})$  both verify with respect to  $\text{com}_{\text{MT}}$ , which by the position-binding property of Merkle commitments, happens with probability at most  $\epsilon_{\text{MT}} = \text{negl}(\lambda)$ .
- $\hat{y}_{i^*} = y'_{i^*}$ : In this case,  $y'_{i^*} = \hat{y}_{i^*} \neq f'(i^*)$ , yet it holds that both  $\text{PC.Verify}(\text{com}_{\text{PC}}, i^*, y'_{i^*}, \pi'_{\text{PC}, i^*}) = 1$ , and  $\text{PC.Verify}(\text{com}_{\text{PC}}, i^*, f'(i^*), \pi_{\text{PC}, i^*}) = 1$ , where  $\pi_{\text{PC}, i^*}$  is the opening proof created with respect to  $\text{com}_{\text{PC}}$  for the polynomial  $f'(X)$  and randomness  $r$ . However, by the evaluation-binding property of the PC, this happens with probability at most  $\epsilon_{\text{PC}} = \text{negl}(\lambda)$ .

Consequently, event 4 happens with probability at most  $\epsilon_{\text{MT}} + \epsilon_{\text{PC}}$ .

Finally, we observe that the extractor outputs over  $(1 - \nu(k))n$  leaf nodes (and Merkle opening proofs) that satisfy the points above except with probability  $\text{negl}(\lambda)$ . This concludes the proof.  $\square$

Defining the adversaries  $\mathcal{A}_1 := (\mathbf{S}_1, \dots, \mathbf{S}_\ell, \mathbf{R}_1)$  and  $\mathcal{A}_2 := (\mathbf{S}_1, \dots, \mathbf{S}_\ell, \mathbf{R}_2)$ , by Lemma 2,  $(\mathbf{S}_1, \dots, \mathbf{S}_\ell)$  and  $\mathbf{R}_1$  together must have queried the random oracle with over  $(1 - \nu(k))n$ -fraction of the leaf nodes before the challenge phase, and except with negligible probability, for over  $(1 - \nu(k))n$  queried leaves  $\{(a_i, \hat{m}_{a_i}, \hat{\pi}_{\text{VC}, a_i}, \hat{y}_{a_i})\}_{i \in [\ell_1]}$ , it holds that

- $\text{MT.Verify}(\text{com}_{\text{MT}}, a_i, (a_i, \hat{m}_{a_i}, \hat{\pi}_{\text{VC}, a_i}, \hat{y}_{a_i}), \hat{\pi}_{\text{MT}, a_i}) = 1$ ,
- $\hat{m}_{a_i} = m_{a_i}$ ,
- $\text{VC.Verify}(\text{com}_{\text{VC}}, a_i, \hat{m}_{a_i}, \hat{\pi}_{\text{VC}, a_i}) = 1$ ,
- $\hat{y}_{a_i} = f'(a_i)$ .

We next construct the extractors  $\text{Ext}^{\mathbf{S}_j}$  and  $\text{Ext}^{\mathbf{R}_1}$ :

$\text{Ext}^{\mathbf{R}_1}(\text{pk}, \text{com}, (\mathbf{R}_1, (\mathbf{S}_1, \dots, \mathbf{S}_\ell),)) \rightarrow m''$ : Unlike in the proof of Lemma 1, an extractor cannot follow a “top-down” approach to arrive at the leaf nodes, which identifies each leaf query. Indeed,  $\mathbf{R}_1$  can for example delegate computation of the intermediate nodes in the Merkle tree without compromising the privacy of its secret key, as in a hiding Merkle commitment, the leaves are committed with randomness. However, when leaf nodes are contained in the random oracle queries made by  $\mathbf{R}_1$ , the extractor can identify them efficiently by relying on the VC proofs contained in the leaf nodes. More formally,  $\text{Ext}^{\mathbf{R}_1}$  does the following:

- Maintain a list  $\mathcal{L}$  of tuples from  $[n] \times \mathcal{M}$  containing the entries of  $m$  that have been recovered so far.
- Parse  $\text{com} = (\text{com}_{\text{MT}}, \text{com}_{\text{PC}}, \text{com}_{\text{VC}})$ .
- Let  $Q_R$  denote the random oracle queries made by  $\mathbf{R}_1$ . Try parsing each query as  $(i, \bar{m}_i, \bar{\pi}_{\text{VC}, i}, \bar{y}_i)$ , and if  $\text{VC.Verify}(\text{com}_{\text{VC}}, i, \bar{m}_i, \bar{\pi}_{\text{VC}, i}) = 1$ , add it to the list  $\mathcal{L} \leftarrow \mathcal{L} \cup \{(i, \bar{m}_i)\}$ . If some  $(i, \bar{m}'_i)$  with  $\bar{m}'_i \neq \bar{m}_i$  already exists in the list, abort.
- Define  $m''$  where for  $i \in [n]$ 

$$m''_i = \begin{cases} \bar{m}_i & \text{if } (i, \bar{m}_i) \in \mathcal{L}, \\ \perp & \text{otherwise} \end{cases}$$
- Output  $m''$ .



By the position-binding property of VC, it is infeasible for the adversary to produce conflicting openings for  $\text{com}_{\text{VC}}$  at the same position  $i$ , and hence  $\text{Ext}^{\mathbf{R}_1}$  does not abort except with negligible probability. Furthermore, it must be that  $m''_{a_i} = m_{a_i}$  for all  $i \in [\ell_1]$ ,  $a_i \in \mathcal{L}$ , except with negligible probability; since (i)  $\hat{m}_{a_i} = m_{a_i}$  for all  $i \in [\ell_1]$ , and, (ii) as for all  $i \in [\ell_1]$ ,  $a_i \in \mathcal{L}$ ,

$$\text{VC.Verify}(\text{com}_{\text{VC}}, a_i, m''_{a_i}, \bar{\pi}_{\text{VC}, a_i}) = \text{VC.Verify}(\text{com}_{\text{VC}}, a_i, \hat{m}_{a_i}, \hat{\pi}_{\text{VC}, a_i}) = 1,$$

by the position-binding property of VC,  $m''_{a_i} = \hat{m}_{a_i}$  for these indices. Recalling that  $\mu(k) = 1 - \nu(k) - k/n$ , if  $|\{i \in [\ell_1] \mid m''_{a_i} = m_{a_i}\}| > (1 - \mu(k))n$ , then  $\mathcal{A}$  loses the  $\mu(k)$ -soundness game. We next show that if this is not the case, then one of  $\{\text{Ext}^{\mathbf{S}_j}\}_{j \in [\ell]}$  can recover the secret key.

$\text{Ext}^{\mathbf{S}_j}(\text{pk}, \text{com}, \langle \mathbf{S}_j, \mathbf{R}_1 \rangle, \langle \mathbf{S}_j, \mathbf{R}_2, \{(m'_i, \pi_i)\}_{i \in \text{chal}} \rangle) \rightarrow \text{sk}$ : This extractor monitors the random oracle queries for valid leaf nodes by  $\mathbf{S}_j$ . If one of them is outside of  $\text{chal}$ , then using the collected  $k+1$  points  $y_i$  output by  $\mathbf{R}_2$ , the interpolating polynomial reveals the secret key.

- Maintain a list  $\mathcal{L}$  of tuples from  $[n] \times \mathcal{M} \times \mathbb{Z}_p$  containing the set of evaluations that have been recovered.
- Parse  $\text{com} = (\text{com}_{\text{MT}}, \text{com}_{\text{PC}}, \text{com}_{\text{VC}})$ .
- For  $i \in \text{chal}$ , parse  $\mathbf{R}_2$ 's output  $m'_i$  and  $\pi_i = (y_i, \pi_{\text{PC}, i}, \pi_{\text{MT}, i}, \pi_{\text{VC}, i}, \pi_{\text{NIZK}, i})$ .
- Let  $Q_S$  denote the random oracle queries made by  $\mathbf{S}_j$ . Try parsing each query as  $(i, \bar{m}_i, \bar{\pi}_{\text{VC}, i}, \bar{y}_i)$ , and if  $i \notin \text{chal}$  and  $\text{VC.Verify}(\text{com}_{\text{VC}}, i, \bar{m}_i, \bar{\pi}_{\text{VC}, i}) = 1$ , add it to the list  $\mathcal{L} \leftarrow \mathcal{L} \cup \{(i, \bar{m}_i, \bar{y}_i)\}$ .
- If  $\mathcal{L} = \emptyset$ , abort.
- Else for all entries  $(i, \bar{m}_i, \bar{y}_i) \in \mathcal{L}$ : Interpolate a polynomial  $f_i(X)$  through the point  $(i, \bar{y}_i)$  as well as  $\{(i, y_i)\}_{i \in \text{chal}}$ . If  $(f_i(0), \text{pk}) \in \mathcal{R}$  then stop and output  $f_i(0)$ .
- Else, abort.

Clearly,  $\text{Ext}^{\mathbf{S}_j}$  runs in polynomial time.

As in the case of  $\mathbf{R}_1$ , by the position-binding property of the vector commitment scheme, the extractor does not abort when adding entries to  $\mathcal{L}$  except with negligible probability. Moreover, as  $(\mathbf{S}_1, \dots, \mathbf{S}_\ell)$  and  $\mathbf{R}_1$  together must have queried the random oracle with the Merkle tree leaves  $\{(a_i, \hat{m}_{a_i}, \hat{\pi}_{\text{VC}, a_i}, \hat{y}_{a_i})\}_{i \in [\ell_1]}$ , where  $\hat{y}_{a_i} = f'(a_i)$  except with negligible probability, if  $|\{i \in [\ell_1] \mid m_{a_i} = \hat{m}_{a_i}\}| \leq (1 - \mu(k))n$ ,  $\mu(k) = 1 - \nu(k) - k/n$ , there exist a sender  $\mathbf{S}_j$  such that one of the polynomials interpolated by  $\mathbf{S}_j$  is  $f_i(X) = f'(X)$ . Thus, for the interpolated polynomial, it holds that  $(f_i(0), \text{pk}) \in \mathcal{R}$ . Hence, if  $\text{Ext}^{\mathbf{R}_1}$  outputs at most  $(1 - \mu(k))n$  points, a value  $\text{sk}$  such that  $(\text{sk}, \text{pk}) \in \mathbf{R}$  can be extracted from  $\mathbf{S}_j$ 's random oracle queries except with negligible probability. This implies that  $\mathcal{A}$  cannot pass the soundness game except with negligible probability (Fig. 1) and concludes the proof showing that the construction in Section 6.1 is a secure non-delegatable commitment scheme.  $\square$

As the proof of Theorem 3 uses the  $q$ -Bilinear Strong Diffie–Hellman assumption, we define it below:

**Definition 10.** The  $q$ -Bilinear Strong Diffie–Hellman assumption, denoted by  $q$ -BSDH and parameterized by  $q \in \mathbb{N}$ , states that for any PPT adversary  $\mathcal{A}$ ,

$$\Pr \left[ t = e(g, h)^{\frac{1}{\tau+c}} \mid \tau \leftarrow \mathbb{Z}_p, (c, t) \leftarrow \mathcal{A}(g, g^\tau, \dots, g^{\tau^q}, h) \right] = \text{negl}(\lambda)$$

for any value of  $c \in \mathbb{Z}_p \setminus \tau$ .

**Theorem 3.** If VC is a computationally position-binding vector commitment scheme, and the  $(k+1)$ -DLOG assumption holds, then the construction in Section 6.2 satisfies non-delegatability with  $\mu(k) = (2^{-\lambda/k} - k/n)$ -soundness in the random oracle model against algebraic adversaries.

*Proof.* Consider a PPT adversary  $\mathcal{A} = (\mathbf{S}_1, \dots, \mathbf{S}_\ell, \mathbf{R}_1, \mathbf{R}_2)$  such that for the messages  $m$ , commitment  $\text{com}$  and proof  $\pi$  output by  $\mathcal{A}$ , it holds that  $m'_i = m_i$  and  $\text{Verify}(\text{pk}, \text{com}, i, m_i, \pi_i) = 1$  for all  $i \in \text{chal}$ . Let  $\text{com} = (\text{com}_{\text{MT}}, \text{com}_{\text{PC}}, \text{com}_{\text{VC}})$  and  $\pi_i = (f(i), \pi_{\text{PC},i}, \pi_{\text{MT},i}, \pi_{\text{VC},i}, \pi_{\text{NIZK}})$ . Again by Lemma 1, we know that  $(\mathbf{S}_1, \dots, \mathbf{S}_\ell)$  and  $\mathbf{R}_1$  together must have queried the random oracle with over  $(1 - \nu(k))$ -fraction of the leaf nodes, which verify with respect to  $\text{com}_{\text{MT}}$ , before the challenge phase, except with probability  $\epsilon_{\text{MT}} = \text{negl}(\lambda)$ . Let  $\{(a_i, \hat{m}_{a_i}, \hat{\pi}_{\text{VC},a_i}, \hat{y}_{a_i})\}_{i \in [\ell_1]}$  denote the queried leaves, where  $\ell_1 > (1 - \nu(k))n$ , and note that  $\text{MT.Verify}(\text{com}_{\text{MT}}, a_i, (a_i, \hat{m}_{a_i}, \hat{\pi}_{\text{VC},a_i}, \hat{y}_{a_i}), \pi'_{\text{MT},a_i}) = 1$  for all  $i \in [\ell_1]$ .

By the extractability of the KZG scheme, a polynomial  $f'(X)$  along with its randomness  $r'$ , can be extracted from the KZG opening proof  $\pi_{\text{KZG},i}$  for some  $i \in \text{chal}$ , using the expected-polynomial-time extractor for the following relation:

$$\{(\text{com}_{\text{PC}}, x, y) \mid \exists (f(X), r) \in \mathbb{Z}_p^{\leq d} \times \{0, 1\}^* : \text{PC.Commit}(f(X); r) = \text{com}_{\text{PC}} \wedge f(x) = y\}$$

Let  $f'(X)$  denote the output of this extractor, and note that  $f'(X) \in \mathbb{Z}_p^{\leq k}[X]$ ,  $\text{com}_{\text{PC}} = \text{PC.Commit}(f'(X); r')$ , and  $f'(i) = f(i)$ .

The rest of the proof proceeds as in the proof of Theorem 2, except that for  $f'(0)$ , we prove that  $(f'(0), \text{pk}) \in \mathcal{R}$  as follows: For contradiction, suppose  $(f'(0), \text{pk}) \notin \mathcal{R}$ , i.e.,  $g^{f'(0)} = \overline{\text{pk}}^1 \neq \text{pk}^1$  with non-negligible probability. Define  $(\pi_{\text{KZG},0}^1, \pi_{\text{KZG},0}^2)$  as the opening proof of the polynomial  $f'(X)$  with randomness  $r'$  at point 0. Recall the proof  $\pi_{\text{KZG},0} = (\pi_{\text{KZG},0}^1, \pi_{\text{KZG},0}^2)$  output as part of the proofs  $\pi_i$ . Then,

$$\begin{aligned} e(\text{com}_{\text{KZG}}/\text{pk}^1, h) &= e(\pi_{\text{KZG},0}^1, h^\tau) \cdot e(\pi_{\text{KZG},0}^2, h^\eta), \\ e(\text{com}_{\text{KZG}}/\overline{\text{pk}}^1, h) &= e(\pi_{\text{KZG},0}^1, h^\tau) \cdot e(\pi_{\text{KZG},0}^2, h^\eta) \end{aligned}$$

By dividing both equations,  $\mathcal{A}$  can obtain;

$$\begin{aligned} e\left(\frac{\overline{\text{pk}}^1}{\text{pk}^1}, h\right) &= e\left(\frac{\pi_{\text{KZG},0}^1}{\pi_{\text{KZG},0}^1}, h^\tau\right) \cdot e\left(\frac{\pi_{\text{KZG},0}^2}{\pi_{\text{KZG},0}^2}, h^\eta\right) \\ \implies e\left(\frac{\overline{\text{pk}}^1}{\text{pk}^1}, h\right) \cdot e\left(\frac{\pi_{\text{KZG},0}^2}{\pi_{\text{KZG},0}^2}, h^\eta\right) &= e\left(\left(\frac{\pi_{\text{KZG},0}^1}{\pi_{\text{KZG},0}^1}\right)^\tau, h\right) \\ \implies e\left(\frac{\pi_{\text{KZG},0}^1}{\pi_{\text{KZG},0}^1}, h\right) \cdot e\left(\frac{\pi_{\text{KZG},0}^2}{\pi_{\text{KZG},0}^2}, h^\eta\right)^{\frac{1}{\tau}} &= e\left(\frac{\overline{\text{pk}}^1}{\text{pk}^1}, h\right)^{\frac{1}{\tau}} \end{aligned}$$

Finally, consider an adversary  $\mathcal{B}$  that receives a powers-of- $\tau$  setup, samples a uniformly random  $\alpha \leftarrow \mathbb{Z}_p$  and a uniformly-random  $\text{sk} \leftarrow \mathbb{Z}_p$ , and runs  $\mathcal{A}$  with  $\text{pk}^1 := g^{\text{sk}}$ ,  $\text{pk}^2 := h^{\text{sk}}$ , and the  $\text{crs}_{\text{KZG}}$  set to be the powers-of- $\tau$  along with  $g^\eta := (g^\tau)^\alpha$  and  $h^\eta := (h^\tau)^\alpha$  (this  $\text{crs}_{\text{KZG}}$  has the same distribution as an honestly generated  $\text{crs}_{\text{KZG}}$ ). Then, for the opening proofs found by  $\mathcal{A}$ , the expression above becomes

$$\begin{aligned} e\left(\frac{\pi_{\text{KZG},0}^1}{\pi_{\text{KZG},0}^1} \cdot \left(\frac{\pi_{\text{KZG},0}^2}{\pi_{\text{KZG},0}^2}\right)^\alpha, h\right) &= e\left(\frac{\overline{\text{pk}}^1}{\text{pk}^1}, h\right)^{\frac{1}{\tau}} \\ \implies e\left(\frac{\pi_{\text{KZG},0}^1}{\pi_{\text{KZG},0}^1} \cdot \left(\frac{\pi_{\text{KZG},0}^2}{\pi_{\text{KZG},0}^2}\right)^\alpha, h\right)^{1/(f'(0)-\text{sk})} &= e(g, h)^{\frac{1}{\tau}}, \end{aligned}$$

where  $\text{sk} \neq f'(0)$  by assumption (which happens with non-negligible probability), and  $\mathcal{B}$  can calculate the left-hand side to find  $e(g, h)^{1/\tau}$ . In other words,  $\mathcal{A}$  breaks  $(k+1)$ -BSDH, which is implied by the  $(k+1)$ -DLOG assumption in AGM [BFL20]. Since this happens with negligible probability, a value  $\text{sk} := f_{\mathcal{L}}(0)$  such that  $(\text{sk}, \text{pk}) \in \mathbf{R}$  can be extracted from  $\mathbf{S}_j$ 's random oracle queries with non-negligible probability.  $\square$